



Information Technology Institute (ITI)

RANPilotAI

AI-Powered RAN Performance
Optimization Assistant

Graduation Project • 2026

Development Team

Mahmoud Ashraf • Ibrahim Samy • Khaled Mogahed
Tasneem Amein • Mohab Tarek

Track Supervisor
Dr. Salma Sobhy

In Collaboration with



Abstract

Modern Radio Access Network (RAN) optimization relies heavily on manual inspection of vast volumes of performance data, expert engineering judgement, and reactive troubleshooting workflows. As LTE and 5G New Radio (NR) networks continue to grow in scale and complexity, this approach becomes increasingly inefficient and unscalable. This graduation project presents **RANPilotAI**, an integrated, AI-powered platform designed to automate and augment the daily workflows of Radio Frequency (RF) optimization engineers in 4G and 5G mobile networks.

The platform unifies four specialized analytical modules within a single web-based environment called the **Telecom Hub**, implemented using the Streamlit framework. Engineers can perform multiple optimization tasks through a common interface without switching between separate software tools.

The first module, the **KPI Degradation Analyzer**, automatically detects performance degradation across 13 Key Performance Indicator categories — including throughput, accessibility, retainability, mobility, availability, VoLTE, CSFB, and RRC re-establishment — using a three-layer analytical pipeline. The pipeline applies Welch’s t-test for statistical confidence scoring, RF-aware root cause analysis with configurable severity weighting, and a data quality framework that handles sentinel values, unit-aware validation, and same-weekday baseline imputation.

The second module, the **KPI Forecasting Engine**, predicts future network performance trends using two complementary time-series models: XGBoost with leakage-safe cross-KPI feature engineering, and Holt-Winters exponential smoothing. The engine provides backtesting metrics (MAE, RMSE, MAPE), STL-based seasonality analysis, recursive 7-day forward forecasts, and residual diagnostics that allow engineers to anticipate degradation before it manifests operationally.

The third module, the **Optimization Action Analyzer**, correlates RF configuration changes — such as antenna tilt and transmit power adjustments — with KPI impact using Dolt, a version-controlled SQL database. The module aligns configuration commit history with KPI time windows using weekday-matched baseline comparison, groups same-day commits into unified action hunks, and presents results through interactive timeline, before/after overlay, delta bar, and summary impact visualizations.

The fourth module, the **Telecom RAG Assistant**, is a Retrieval-Augmented Generation system that enables engineers to query official 3GPP Technical Specifications using natural language. The system combines section-aware PDF chunking, BGE-large dense embeddings, local Qdrant vector search, cross-encoder re-ranking, and a strict anti-hallucination prompt to produce traceable, citation-backed answers grounded directly in official standards documentation.

Experimental results demonstrate that the integrated platform substantially reduces the time required for network performance analysis, root cause identification, and standards consultation, while maintaining engineering accuracy and traceability. The RAN-PilotAI platform is implemented entirely in Python 3.11 and is accessible through a standard web browser without additional client-side installation.

Dedication

To our families,

*whose patience, encouragement, and unwavering belief sustained us
through every late night and every challenge this project brought.*

To our instructors and supervisors at the Information Technology Institute,

*whose guidance shaped not only this work
but the engineers we have become.*

*And to every RF engineer who spends hours
searching through performance counters and specification pages —
this platform was built for you.*

Acknowledgements

We would like to express our sincere gratitude to all those who contributed to the successful completion of this graduation project.

First and foremost, we extend our deepest appreciation to the **Information Technology Institute (ITI)**, whose rigorous academic programme and practical engineering environment provided the foundation upon which this work was built. The resources, facilities, and opportunities made available by the institute were essential to the realization of the RANPilotAI platform.

We are especially grateful to our **supervisors and instructors** for their continuous guidance, technical insight, and encouragement throughout the project lifecycle. Their expertise in telecommunications engineering, software development, and artificial intelligence significantly influenced the direction and quality of this work.

We also thank our colleagues and peers at ITI for their constructive feedback, collaborative spirit, and shared knowledge throughout the duration of the programme.

On a personal level, each team member would like to acknowledge the support of their family and friends, whose patience and encouragement made it possible to devote the time and effort this project demanded.

Finally, we acknowledge the open-source communities behind the tools and libraries that made this platform possible, including Streamlit, XGBoost, Qdrant, Sentence Transformers, Dolt, and the broader Python scientific ecosystem. This project stands on the shoulders of the open-source work that preceded it.

The RANPilotAI Team

- Mahmoud Ashraf
- Ibrahim Samy
- Khaled Mogahed
- Tasneem Amein
- Mohab Tarek

Contents

Abstract	i
Dedication	ii
Acknowledgements	iii
1 Introduction	1
2 Literature Review	2
3 Overall System Design	3
4 KPI Degradation Analyzer	4
5 LTE KPI Forecasting Engine	5
6 Optimization Action Analyzer	6
7 Telecom RAG Assistant	7
7.1 Introduction	7
7.2 Motivation	9
7.3 Problem Statement	9
7.4 Background	10
7.4.1 Large Language Models (LLMs)	10
7.4.2 Retrieval-Augmented Generation (RAG)	11
7.4.3 Embedding Models	11
7.4.4 Vector Databases	12
7.4.5 Cross Encoder Re-ranking	12
7.5 Overall System Architecture	12
7.6 Software Architecture	14
7.7 Query Processing Workflow	15
7.8 PDF Processing	16
7.9 Section-aware Chunking	16
7.10 Embedding Generation	17
7.11 Vector Database (Qdrant)	17
7.12 Metadata Filtering	18
7.13 Hybrid Retrieval Pipeline	19
7.14 Cross Encoder Re-ranking	20
7.15 Prompt Construction	21
7.16 LLM Response Generation	21

7.17	User Interface	22
7.18	Performance Evaluation	25
7.19	Discussion	26
7.20	Limitations	26
7.21	Future Improvements	27
7.22	Chapter Summary	27
8	System Integration	28
8.1	Introduction	28
8.2	Overall Platform Architecture	28
8.3	Integration Objectives	29
8.4	Layered Software Architecture	30
8.5	Module Integration	31
8.6	Shared Components	33
8.7	Data Flow Between Modules	34
8.8	User Navigation Workflow	35
8.9	Deployment Environment	36
8.10	Platform Scalability	37
8.11	Integration Challenges	38
8.12	Future Extensions	38
8.13	Chapter Summary	39
9	Experimental Results	40
10	Conclusion and Future Work	41
10.1	Conclusion	41
10.2	Limitations	43
10.3	Future Work	43
10.4	Closing Remarks	44
11	User Manual	46
11.1	System Requirements	46
11.2	Installation	46
11.2.1	Step 1 — Install Python	46
11.2.2	Step 2 — Install Dependencies	46
11.2.3	Step 3 — Configure the RAG Assistant (First Time Only)	47
11.2.4	Step 4 — Configure the Optimization Action Analyzer (If Required)	47
11.3	Launching the Platform	47
11.4	Navigating the Telecom Hub	48
11.5	Tool 1 — KPI Degradation Analyzer	48
11.5.1	Purpose	48
11.5.2	Input Data Format	48
11.5.3	Step-by-Step Usage	48
11.5.4	Understanding the Output	49
11.6	Tool 2 — KPI Forecasting Engine	49
11.6.1	Purpose	49
11.6.2	Input Data Format	49
11.6.3	Step-by-Step Usage	50
11.7	Tool 3 — Optimization Action Analyzer	50

11.7.1	Purpose	50
11.7.2	Prerequisites	50
11.7.3	Step-by-Step Usage	51
11.8	Tool 4 — Telecom RAG Assistant	51
11.8.1	Purpose	51
11.8.2	Prerequisites	51
11.8.3	Step-by-Step Usage	51
11.8.4	Performance Tuning	52
11.9	Troubleshooting	52
References		53

List of Figures

7.1	Main interface of the Telecom RAG Assistant integrated into the RAN-PilotAI platform. The interface provides access to the Telecom RAG module, allows filtering by specific 3GPP Technical Specifications, and supports interactive engineering question answering using Retrieval-Augmented Generation.	8
7.2	Overall architecture of the proposed Telecom RAG Assistant.	13
7.3	Real-time execution stages of the Retrieval-Augmented Generation pipeline during query processing.	15
7.4	Specification Filter allowing users to restrict semantic retrieval to selected 3GPP Technical Specifications.	19
7.5	Execution stages of the hybrid Retrieval-Augmented Generation pipeline. .	20
7.6	Generated engineering response for the LTE Event A3 measurement event.	22
7.7	Main interface of the Telecom RAG Assistant.	23
7.8	Context-aware response generated by the Telecom RAG Assistant using information from previous user interactions within the same conversation. .	24
7.9	Structured engineering summary generated by the Telecom RAG Assistant.	24
7.10	Generated response explaining the Packet Data Convergence Protocol (PDCP).	25
7.11	Structured engineering summary for the PDCP protocol.	25
7.12	Pipeline profiling and execution statistics generated during query processing.	26
8.1	Overall architecture of the integrated RANPilotAI platform. The platform groups all analytical modules within a unified engineering environment while providing common platform services shared across the developed applications.	29
8.2	Layered software architecture of the RANPilotAI platform. The platform is organized into four logical layers separating presentation, application logic, engineering modules, and data management.	31
8.3	Module interaction within the RANPilotAI platform. The Telecom Hub coordinates user requests while common software services are shared among all analytical modules.	32
8.4	Component organization of the RANPilotAI platform showing the relationship between the Telecom Hub, shared software components, platform resources, and engineering modules.	34
8.5	Overall data flow within the RANPilotAI platform. All requests are coordinated by the Telecom Hub before reaching the selected engineering module.	35
8.6	Typical user navigation workflow within the RANPilotAI platform.	36

List of Tables

Chapter 1

Introduction

Write the Introduction here.

Chapter 2

Literature Review

Write the Literature Review here.

Chapter 3

Overall System Design

Write the System Design here.

Chapter 4

KPI Degradation Analyzer

Write the chapter here.

Chapter 5

LTE KPI Forecasting Engine

Write the chapter here.

Chapter 6

Optimization Action Analyzer

Write the chapter here.

Chapter 7

Telecom RAG Assistant

The Telecom Retrieval-Augmented Generation (RAG) Assistant is one of the four intelligent modules developed within the RANPilotAI platform. It is designed to assist Radio Access Network (RAN) engineers in efficiently retrieving and understanding information from official Third Generation Partnership Project (3GPP) Technical Specifications. Unlike conventional AI chatbots that rely solely on the internal knowledge of Large Language Models (LLMs), the proposed assistant integrates semantic retrieval with language generation, ensuring that every generated response is grounded in authentic engineering documentation.

The assistant combines document preprocessing, semantic chunking, dense embedding generation, vector similarity search, neural re-ranking, prompt engineering, and Large Language Models into a unified Retrieval-Augmented Generation pipeline. Consequently, engineers can ask technical questions using natural language while receiving structured responses supported by direct citations from the corresponding 3GPP specifications.

7.1 Introduction

Modern LTE and 5G New Radio (NR) networks are standardized by hundreds of Technical Specifications published by the Third Generation Partnership Project (3GPP). These specifications define every aspect of the cellular system, including Radio Resource Control (RRC), Packet Data Convergence Protocol (PDCP), Radio Link Control (RLC), Medium Access Control (MAC), mobility management, measurement procedures, bearer configuration, security mechanisms, and radio resource management.

Although these documents serve as the authoritative engineering reference, they collectively contain thousands of pages distributed across multiple releases and specification documents. Locating a specific protocol definition, signaling procedure, configuration parameter, or ASN.1 structure often requires manually searching several PDF files and navigating numerous sections before identifying the required information. Traditional keyword-based searching is inefficient because engineering concepts are frequently distributed across different chapters, tables, annexes, and protocol descriptions.

Recent advances in Large Language Models (LLMs) have significantly improved natural language interaction with technical information. However, general-purpose language models cannot be considered authoritative engineering references because they generate responses solely from learned knowledge and may therefore produce inaccurate or hallucinated technical information. Such inaccuracies are unacceptable in professional telecommunication environments where engineering decisions must comply with official

standards.

To address these challenges, the proposed Telecom RAG Assistant employs a Retrieval-Augmented Generation architecture. Rather than generating answers directly from the language model, the system first retrieves the most relevant sections from locally indexed 3GPP specifications before constructing the prompt supplied to the LLM. This approach combines the reasoning capabilities of modern language models with the reliability of official technical documentation, producing responses that are both accurate and traceable.

The Telecom RAG Assistant is fully integrated into the RANPilotAI platform alongside the KPI Degradation Analyzer, KPI Forecaster, and Optimization Action Analyzer. The platform provides a unified environment in which engineers can perform network analysis and standards consultation without switching between multiple software applications.

Figure 7.1 presents the main interface of the Telecom RAG Assistant. The interface enables engineers to select the RAG module, optionally restrict the search to a specific Technical Specification, submit engineering questions using natural language, and receive structured answers supported by citations extracted from the original specifications.

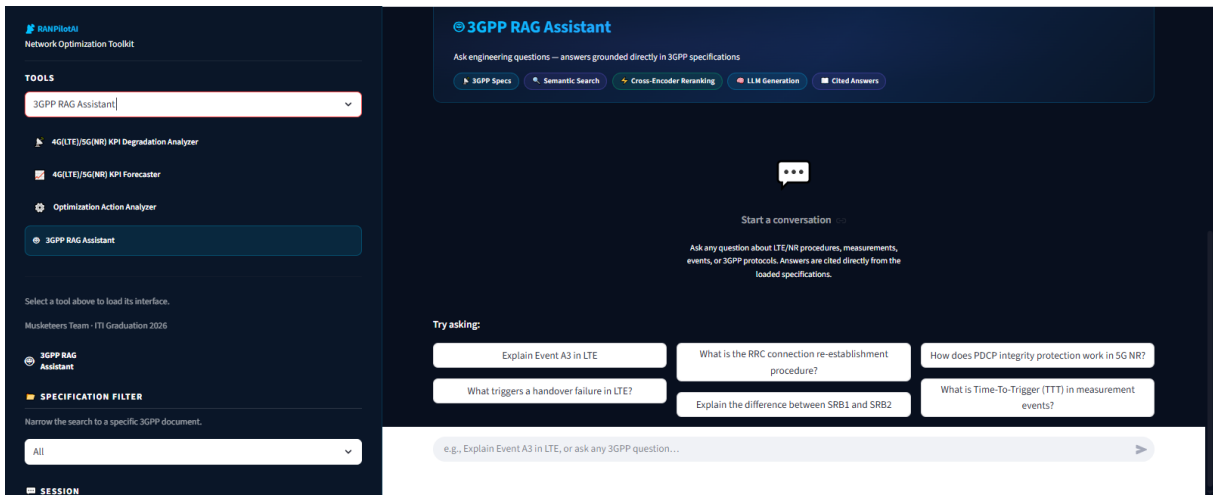


Figure 7.1: Main interface of the Telecom RAG Assistant integrated into the RANPilotAI platform. The interface provides access to the Telecom RAG module, allows filtering by specific 3GPP Technical Specifications, and supports interactive engineering question answering using Retrieval-Augmented Generation.

Unlike conventional document search systems, the developed assistant performs semantic understanding of both the user’s query and the indexed specification content. Consequently, relevant information can be retrieved even when the wording used by the engineer differs from the terminology employed within the official specification.

Furthermore, the generated responses are presented in a structured and readable format that includes hierarchical headings, explanatory paragraphs, engineering tables, bullet lists, and expandable citation panels. This presentation enables engineers not only to obtain concise explanations but also to verify every statement against the corresponding sections of the official 3GPP documentation.

7.2 Motivation

Radio Access Network engineers routinely consult 3GPP Technical Specifications during network optimization, protocol analysis, interoperability testing, feature implementation, and troubleshooting activities. As LTE and 5G technologies continue to evolve, the complexity of these specifications has increased substantially, introducing thousands of configurable parameters, signaling procedures, optional features, and release-specific enhancements.

In traditional engineering workflows, locating the required information often involves opening multiple PDF documents, performing repetitive keyword searches, comparing several specification releases, and manually interpreting protocol descriptions before obtaining a complete understanding of the desired concept. This process is both time-consuming and inefficient, particularly when information is distributed across multiple sections or Technical Specifications.

Although modern Large Language Models provide an intuitive conversational interface, they cannot guarantee engineering accuracy because they do not inherently reference official standards during inference. Consequently, generated responses may contain incomplete explanations, outdated information, or fabricated protocol behavior. Such limitations significantly reduce their reliability for standards-based engineering applications.

The primary motivation behind developing the Telecom RAG Assistant was therefore to combine the natural language understanding capabilities of modern LLMs with the reliability of official 3GPP specifications. By integrating semantic retrieval with language generation, the assistant ensures that every response is generated using authentic engineering context retrieved directly from the indexed documents.

Another important objective was to reduce the time required for standards consultation. Instead of manually navigating hundreds of pages across multiple specifications, engineers can simply submit questions in natural language and immediately receive concise technical explanations accompanied by explicit references to the corresponding Technical Specification, release version, section number, and page number.

The developed graphical interface further enhances usability by allowing users to restrict searches to specific Technical Specifications through the Specification Filter, observe the internal execution stages of the Retrieval-Augmented Generation pipeline, and review supporting citations for every generated response. This transforms static technical documentation into an interactive engineering assistant capable of supporting daily telecommunication engineering activities.

7.3 Problem Statement

The exponential growth of LTE and 5G New Radio (NR) standards has significantly increased the complexity of engineering documentation. Modern 3GPP Technical Specifications consist of thousands of pages describing protocol procedures, signaling messages, configuration parameters, ASN.1 definitions, state machines, timers, and implementation guidelines. Although these documents serve as the official reference for telecommunication systems, efficiently locating the required information remains a major challenge for network engineers.

Traditional document retrieval techniques rely primarily on keyword-based searches within Portable Document Format (PDF) files. While these methods are suitable for

locating exact phrases, they frequently fail when users employ different terminology or when the required information is distributed across multiple specification sections. For example, an engineer searching for "Why is the HO-Conn field required?" may not retrieve the relevant specification if the document describes the same concept using formal protocol terminology rather than the user's wording.

General-purpose Large Language Models provide an alternative approach by enabling natural language interaction with technical knowledge. However, these models generate responses based on their internal parameters rather than official engineering references. Consequently, they may produce inaccurate explanations, outdated information, or fabricated protocol descriptions. Such hallucinations are unacceptable in professional telecommunication environments where engineering decisions must strictly comply with standardized specifications.

Therefore, the primary challenge addressed in this work is the development of an intelligent question-answering system capable of combining semantic understanding with authoritative document retrieval. The system should allow engineers to interact with 3GPP specifications using natural language while ensuring that every generated response remains grounded in authentic engineering documentation.

To address this challenge, the proposed Telecom RAG Assistant employs a Retrieval-Augmented Generation (RAG) architecture that retrieves the most relevant specification sections before generating the final response. This approach significantly improves factual accuracy while preserving complete traceability to the original Technical Specifications.

7.4 Background

The Telecom RAG Assistant integrates several modern Artificial Intelligence and Information Retrieval technologies to transform static engineering documentation into an interactive knowledge assistant. This section introduces the fundamental concepts that form the basis of the proposed system.

7.4.1 Large Language Models (LLMs)

Large Language Models (LLMs) are deep neural networks trained on massive textual datasets to predict the next token in a sequence [1]. Recent models such as GPT, LLaMA, Claude, Gemini, and DeepSeek have demonstrated remarkable capabilities in natural language understanding, reasoning, summarization, translation, and conversational question answering. These capabilities enable users to interact with complex technical information using natural language instead of predefined commands or structured queries.

Despite their impressive performance, LLMs suffer from several limitations when applied to engineering applications. Since their knowledge is acquired during training, newly published standards or updated technical documents cannot be accessed unless the model is retrained. Furthermore, the generated responses may contain hallucinations, where the model produces fluent but factually incorrect information. In standards-based domains such as telecommunications, such inaccuracies may lead to incorrect engineering decisions.

Consequently, rather than relying exclusively on an LLM, the proposed system augments the language model with external knowledge retrieved directly from official 3GPP specifications.

7.4.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a hybrid Artificial Intelligence framework that combines document retrieval with neural language generation. Instead of generating responses solely from the model’s internal knowledge, a RAG system first retrieves the most relevant documents from an external knowledge base and then incorporates these documents into the prompt supplied to the language model [2].

The general RAG workflow consists of four sequential stages:

1. User query processing.
2. Semantic retrieval of relevant document chunks.
3. Prompt construction using the retrieved context.
4. Response generation by the Large Language Model.

This architecture provides several important advantages over standalone language models. First, it allows the knowledge base to be updated without retraining the language model. Second, it significantly reduces hallucinations because responses are generated using authentic supporting documents. Finally, it enables explicit citation of the retrieved sources, thereby improving transparency and user confidence.

Within the proposed Telecom RAG Assistant, Retrieval-Augmented Generation serves as the core framework that connects semantic search with natural language generation, enabling accurate engineering question answering over official 3GPP Technical Specifications.

7.4.3 Embedding Models

Semantic retrieval requires transforming textual information into numerical representations that preserve contextual meaning. This transformation is performed using embedding models, which encode sentences or document passages into dense high-dimensional vectors.

Unlike traditional keyword matching, embedding models capture semantic similarity between different textual expressions. Consequently, two sentences describing the same engineering concept using different terminology are represented by nearby vectors in the embedding space.

For example, the following engineering questions express similar meanings despite using different wording:

- What is Event A3?
- Explain the A3 measurement event.
- How does LTE Event A3 trigger handover?

A high-quality embedding model generates similar vector representations for these queries, enabling the retrieval engine to identify the same specification sections regardless of the exact wording used by the engineer.

The Telecom RAG Assistant employs a transformer-based embedding model to encode both the indexed specification chunks and incoming user queries into a shared semantic vector space.

7.4.4 Vector Databases

After generating dense embeddings for every document chunk, an efficient mechanism is required to store and retrieve these vectors. Traditional relational databases are not designed for similarity search in high-dimensional vector spaces. Instead, specialized vector databases perform Approximate Nearest Neighbor (ANN) search to efficiently retrieve semantically similar vectors [3].

The proposed system utilizes Qdrant as the vector database responsible for storing all embedded document chunks together with their associated metadata. Besides the dense embeddings, each indexed record contains additional information including:

- Technical Specification number.
- 3GPP Release version.
- Section identifier.
- Page number.
- Document title.

This metadata enables the system to filter search results according to user-selected specifications while preserving complete traceability to the original engineering documents.

7.4.5 Cross Encoder Re-ranking

Although dense vector retrieval effectively identifies semantically related document chunks, the retrieved results may still include partially relevant passages. To improve retrieval precision, the proposed system performs a second-stage neural re-ranking process using a Cross Encoder model.

Unlike embedding models that independently encode queries and documents, Cross Encoders jointly process both inputs and compute a direct relevance score for each query-document pair. This enables the model to perform deeper semantic analysis and produce a more accurate ranking of candidate document chunks.

After the initial vector search retrieves the top candidate passages, the Cross Encoder evaluates each retrieved chunk, assigns a relevance score, and reorders the results accordingly. Only the highest-ranked chunks are subsequently included in the final prompt supplied to the language model.

This two-stage retrieval strategy combines the efficiency of vector similarity search with the accuracy of neural relevance estimation, significantly improving the quality of the generated engineering responses while minimizing irrelevant contextual information.

7.5 Overall System Architecture

The Telecom RAG Assistant follows a modular Retrieval-Augmented Generation (RAG) architecture specifically designed for querying official 3GPP Technical Specifications. The architecture integrates document preprocessing, semantic indexing, vector retrieval, neural re-ranking, prompt construction, and Large Language Model (LLM) inference into a unified pipeline. Each module performs a dedicated task while interacting with adjacent

modules through well-defined interfaces, resulting in a scalable and maintainable software architecture.

Unlike conventional document search systems, the proposed architecture performs semantic retrieval instead of keyword matching. Both the indexed specification documents and incoming user queries are represented as dense embedding vectors. Similarity search is then performed within the embedding space, enabling the retrieval of conceptually related information even when different terminology is used.

The overall architecture of the proposed Telecom RAG Assistant is illustrated in Figure 7.2.

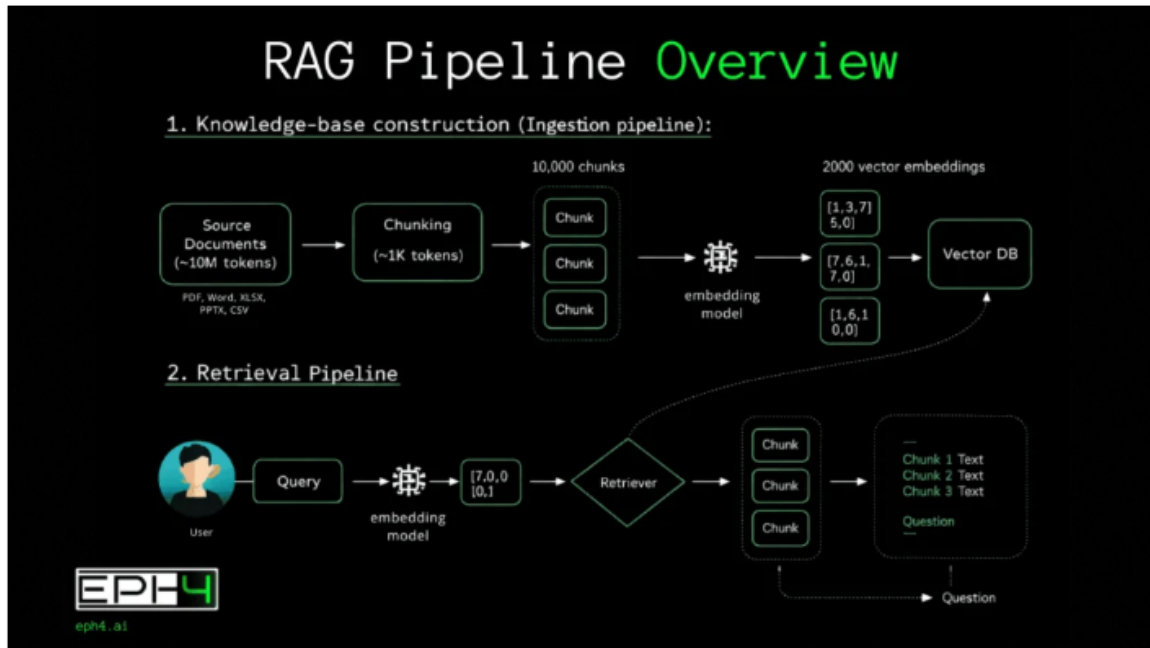


Figure 7.2: Overall architecture of the proposed Telecom RAG Assistant.

The complete system consists of the following processing stages:

1. PDF Processing
2. Section-aware Chunking
3. Embedding Generation
4. Vector Storage using Qdrant
5. Metadata Filtering
6. Dense Vector Retrieval
7. Cross Encoder Re-ranking
8. Prompt Construction
9. LLM Response Generation
10. Response Rendering with Source Citations

Each module contributes to improving retrieval accuracy, reducing hallucination, and maintaining traceability to the original engineering documentation.

7.6 Software Architecture

The Telecom RAG Assistant was implemented using a modular software architecture in which every component is responsible for a specific stage of the Retrieval-Augmented Generation pipeline. This modular design simplifies maintenance, facilitates future extensions, and allows individual components to be optimized independently.

The software architecture is composed of the following modules:

- **PDF Processing Module**

Extracts textual content and metadata from official 3GPP Technical Specifications.

- **Section-aware Chunking Module**

Splits the extracted documents into semantically meaningful chunks while preserving the hierarchical structure of the specifications.

- **Embedding Module**

Generates dense vector representations for every document chunk using the selected embedding model.

- **Vector Database Module**

Stores all embedding vectors together with their associated metadata using the Qdrant vector database.

- **Metadata Filtering Module**

Restricts document retrieval according to user-selected Technical Specifications.

- **Retrieval Module**

Performs semantic similarity search to retrieve the Top-K most relevant document chunks.

- **Cross Encoder Module**

Re-ranks the retrieved chunks according to their semantic relevance to the user query.

- **Prompt Construction Module**

Builds the final prompt by combining the retrieved specification context with the user's question.

- **Large Language Model Module**

Generates the final engineering response using the constructed prompt.

- **Streamlit User Interface**

Provides the graphical interface through which users interact with the complete system.

The modular organization allows each subsystem to operate independently while contributing to the overall Retrieval-Augmented Generation workflow.

7.7 Query Processing Workflow

The complete query processing workflow begins when the engineer submits a natural-language question through the Streamlit interface. Instead of directly forwarding the request to the Large Language Model, the proposed system executes a sequence of retrieval and validation stages to ensure that the generated response is based exclusively on relevant sections extracted from the indexed 3GPP specifications.

Figure 7.3 illustrates the execution stages displayed by the system while processing a user query.

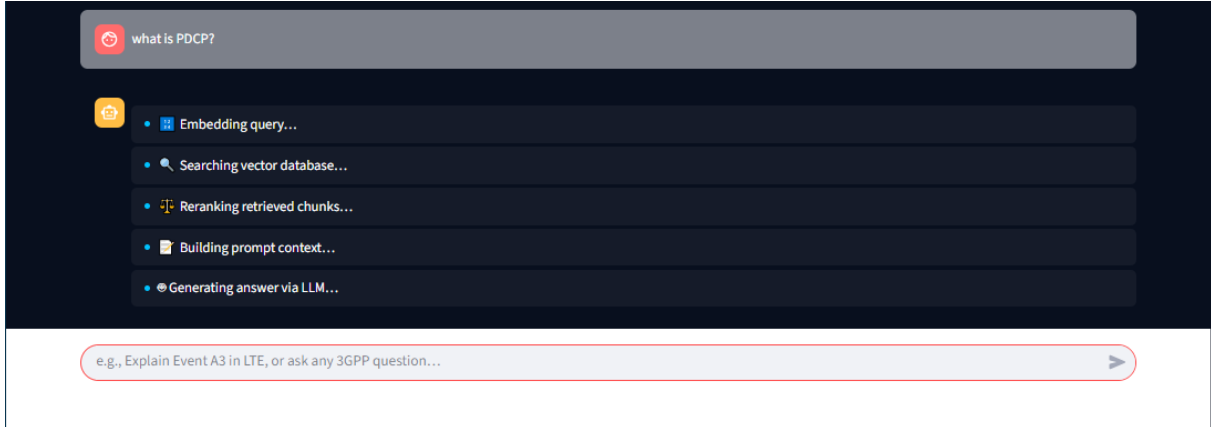


Figure 7.3: Real-time execution stages of the Retrieval-Augmented Generation pipeline during query processing.

After receiving a user query, the system performs the following sequence of operations:

1. The user submits a technical question through the graphical interface.
2. The query is converted into a dense semantic embedding using the embedding model.
3. The generated embedding is submitted to the Qdrant vector database to retrieve the Top-K semantically similar document chunks.
4. Metadata filtering is applied whenever the user restricts the search to a specific Technical Specification.
5. Duplicate chunks are removed to maximize contextual diversity.
6. A Cross Encoder model evaluates each retrieved chunk and computes semantic relevance scores.
7. The highest-ranked chunks are selected to construct the final prompt.
8. The prompt is transmitted to the selected Large Language Model through the Open-Router API.
9. The language model generates a structured engineering response based on the retrieved specification context.
10. Supporting citations are attached to the generated response, enabling engineers to verify every statement directly from the corresponding 3GPP Technical Specification.

The graphical interface provides users with real-time feedback regarding the execution status of every processing stage, including embedding generation, vector retrieval, neural re-ranking, prompt construction, and response generation, thereby improving transparency and user confidence in the retrieval process.

7.8 PDF Processing

The first stage of the proposed Telecom RAG Assistant consists of processing the official 3GPP Technical Specification documents. Since these specifications are distributed as Portable Document Format (PDF) files, their contents must first be transformed into a machine-readable representation before they can be indexed and queried.

Unlike conventional document parsers that simply extract plain text, the proposed system performs structured document analysis in order to preserve the hierarchical organization of the specifications. This is particularly important because 3GPP documents contain numbered sections, subsections, engineering tables, ASN.1 definitions, mathematical expressions, figures, and annexes, all of which contribute to the semantic meaning of the document.

The PDF processing module is responsible for extracting textual content, removing unnecessary formatting artifacts, identifying section boundaries, extracting engineering tables, and collecting document metadata such as the Technical Specification number, release version, page number, and document title. These metadata elements are preserved throughout the indexing pipeline and later attached to every retrieved chunk, allowing engineers to trace generated responses back to their original source.

The implementation utilizes the PyMuPDF library to efficiently parse PDF documents while preserving page-level information. During preprocessing, repetitive page headers, footers, and formatting artifacts are removed in order to improve retrieval quality. Engineering tables are detected and converted into Markdown representations, enabling the language model to interpret structured information more accurately than raw PDF text. This preprocessing stage significantly improves the readability and semantic consistency of the indexed documents.

The processed output generated by this stage serves as the input to the Section-aware Chunking module, which further organizes the extracted content into semantically meaningful units suitable for embedding generation.

7.9 Section-aware Chunking

One of the most important contributions of the proposed Telecom RAG Assistant is the implementation of a section-aware chunking strategy specifically designed for 3GPP Technical Specifications.

Traditional Retrieval-Augmented Generation systems generally divide documents into fixed-size chunks based on a predefined number of tokens or characters. Although computationally simple, this approach frequently separates logically connected engineering concepts across multiple chunks. As a consequence, important contextual information may be lost, leading to incomplete retrieval results and reduced response quality.

To overcome this limitation, the developed system performs hierarchical chunking based on the structural organization of 3GPP specifications. Instead of splitting documents at arbitrary token boundaries, the chunking algorithm detects section headers,

subsection identifiers, engineering tables, figure descriptions, annexes, ASN.1 definitions, and mathematical expressions. Each chunk therefore represents a complete engineering concept rather than an arbitrary text fragment.

- The textual content extracted from the specification.
- Metadata describing the source Technical Specification, release version, section identifier, page number, and document type.

This structured representation enables subsequent retrieval stages to perform semantic search while simultaneously preserving complete traceability to the original engineering documentation.

7.10 Embedding Generation

After document chunking has been completed, every chunk is transformed into a dense numerical representation using a transformer-based embedding model. These dense vectors capture the semantic meaning of each engineering concept, enabling similarity search beyond simple keyword matching.

The embedding model projects both specification chunks and incoming user queries into the same high-dimensional semantic vector space. Consequently, conceptually related texts are positioned close to one another regardless of the exact wording used. This capability enables engineers to formulate questions using natural language while still retrieving the appropriate specification sections.

During the indexing process, each document chunk is independently encoded before being inserted into the vector database. Similarly, every user query submitted through the graphical interface is encoded using the same embedding model. Since both document embeddings and query embeddings share the same semantic space, similarity can be efficiently computed using vector distance metrics.

The generated embeddings constitute the primary index used by the retrieval engine and form the foundation of the semantic search capabilities implemented within the Telecom RAG Assistant.

7.11 Vector Database (Qdrant)

Following embedding generation, all dense vectors are stored inside a local Qdrant vector database. Qdrant was selected due to its efficient Approximate Nearest Neighbor (ANN) search algorithms, support for metadata filtering, lightweight deployment model, and seamless integration with Python-based machine learning frameworks.

Unlike cloud-hosted vector databases, the developed system employs Qdrant in embedded local mode. Consequently, the complete vector database resides on the local machine, eliminating external dependencies while improving data privacy and reducing deployment complexity.

Each indexed record stored within Qdrant contains both the embedding vector and its associated metadata. Typical metadata include:

- Technical Specification number
- 3GPP Release version

- Section identifier
- Page number
- Document title

The inclusion of metadata enables advanced filtering operations during retrieval while preserving direct references to the original engineering documentation.

When a user submits a query, the corresponding embedding vector is generated and compared against all indexed vectors stored in Qdrant. The database retrieves the Top-K most semantically similar chunks, which are subsequently passed to the Cross Encoder re-ranking stage for further refinement.

7.12 Metadata Filtering

Although semantic retrieval effectively identifies document chunks that are conceptually related to the user’s query, searching across all indexed specifications may introduce unnecessary or less relevant results. Since the Telecom RAG Assistant indexes multiple 3GPP Technical Specifications spanning different releases and protocol layers, restricting the search scope can significantly improve retrieval precision.

To address this challenge, the proposed system incorporates a metadata filtering stage before the final retrieval process. Every indexed document stored in the vector database contains descriptive metadata extracted during the indexing stage, including the Technical Specification (TS) number, release version, section identifier, page number, and document title. These metadata fields enable the retrieval engine to perform constrained searches according to the user’s selected criteria.

The graphical user interface provides an interactive Specification Filter that allows engineers to limit the search space to one or more Technical Specifications before submitting a query. For example, an engineer investigating Radio Resource Control (RRC) procedures may restrict the search to TS 36.331, while another engineer interested in Physical Layer procedures may instead search within TS 36.214.

Figure 7.4 illustrates the Specification Filter implemented within the Telecom RAG Assistant.

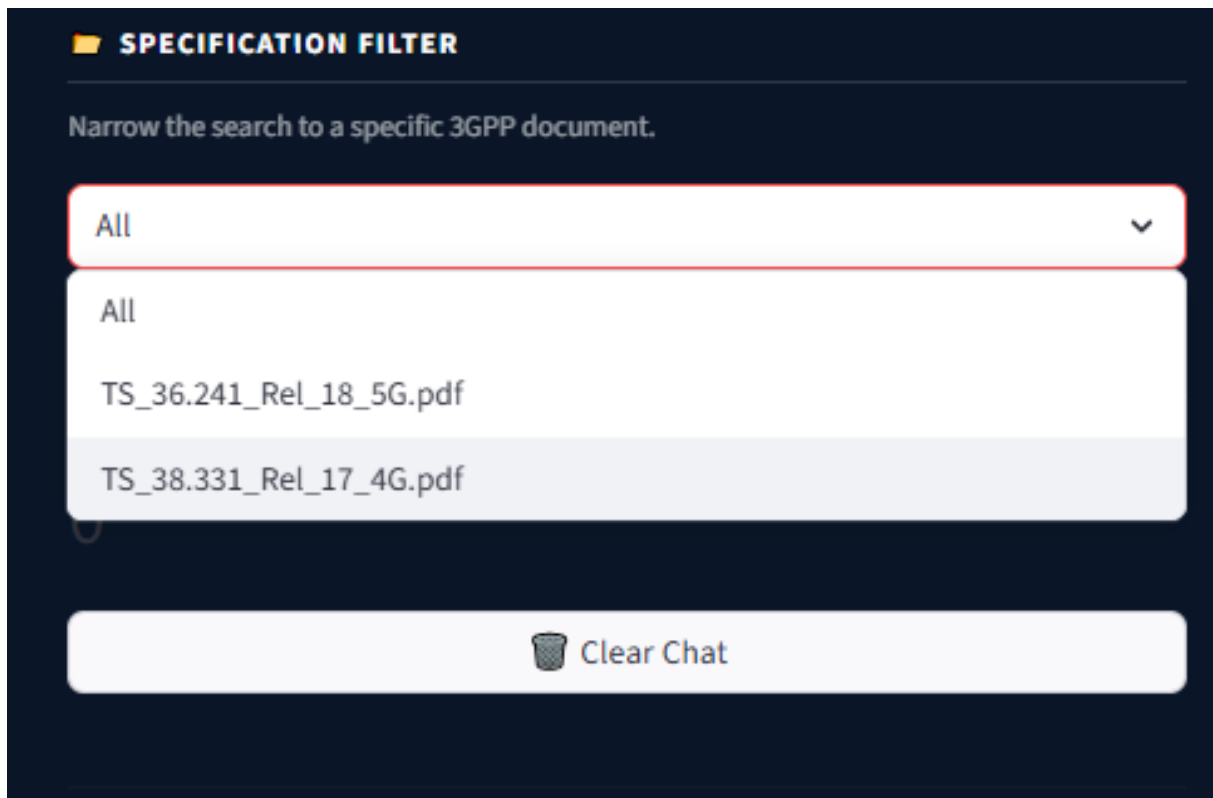


Figure 7.4: Specification Filter allowing users to restrict semantic retrieval to selected 3GPP Technical Specifications.

By eliminating unrelated specifications before semantic retrieval, metadata filtering reduces computational complexity, improves retrieval precision, and minimizes the likelihood of retrieving technically similar but contextually unrelated document sections. This filtering mechanism is particularly valuable when multiple specifications describe related procedures using similar terminology.

7.13 Hybrid Retrieval Pipeline

After applying metadata filtering, the system performs semantic retrieval using a hybrid pipeline that combines dense vector search with neural relevance estimation. This two-stage retrieval strategy balances computational efficiency with retrieval accuracy and forms one of the key engineering contributions of the developed assistant.

Initially, the user’s natural-language question is converted into a dense embedding vector using the same embedding model employed during document indexing. The generated embedding is then submitted to the Qdrant vector database, which performs Approximate Nearest Neighbor (ANN) search to retrieve the Top-K semantically similar document chunks.

Instead of directly forwarding these retrieved chunks to the language model, the proposed system performs an additional refinement stage to improve retrieval quality. This refinement reduces redundant context, removes partially relevant passages, and prioritizes document chunks that best answer the user’s question.

Figure 7.5 presents the execution stages displayed during query processing.

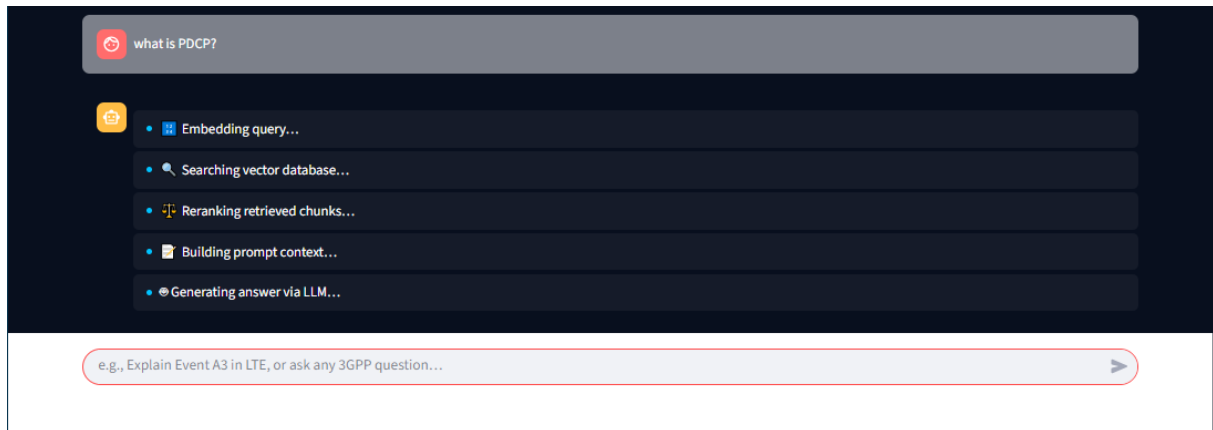


Figure 7.5: Execution stages of the hybrid Retrieval-Augmented Generation pipeline.

The implemented retrieval pipeline consists of the following stages:

1. Query embedding generation.
2. Metadata-aware vector retrieval using Qdrant.
3. Duplicate chunk removal.
4. Cross Encoder semantic re-ranking.
5. Selection of the highest-ranked document chunks.
6. Prompt construction.
7. Large Language Model inference.

This hybrid retrieval strategy significantly improves response quality compared to relying solely on dense vector similarity.

7.14 Cross Encoder Re-ranking

Although dense vector retrieval effectively retrieves semantically related specification sections, the retrieved candidate chunks may still contain partially relevant information. To improve retrieval precision, the Telecom RAG Assistant incorporates a Cross Encoder re-ranking stage before prompt construction.

Unlike embedding models that independently encode the query and document, a Cross Encoder jointly processes both inputs within the same neural network. This architecture enables the model to analyze complex semantic relationships between the user query and each retrieved document chunk, producing a highly accurate relevance score.

Following the initial retrieval stage, every candidate chunk retrieved from Qdrant is evaluated by the Cross Encoder. The model computes an independent relevance score for every query-document pair, after which the retrieved chunks are sorted according to their semantic relevance.

Only the highest-ranked chunks are selected for inclusion within the final prompt supplied to the language model. Consequently, the prompt contains highly relevant engineering context while excluding redundant or weakly related information.

This additional neural re-ranking stage substantially improves the factual accuracy of the generated responses by ensuring that the language model receives only the most informative specification excerpts.

7.15 Prompt Construction

Prompt construction represents the final stage before invoking the Large Language Model. Rather than forwarding only the user’s question, the system constructs a structured prompt that combines the retrieved specification context with carefully designed system instructions.

The prompt consists of three principal components:

1. A system prompt defining the assistant’s behavior and restricting responses to the retrieved engineering context.
2. The highest-ranked specification chunks obtained after semantic retrieval and Cross Encoder re-ranking.
3. The user’s original engineering question.

The system prompt explicitly instructs the language model to generate responses exclusively from the supplied specification excerpts. Whenever the retrieved context does not contain sufficient information to answer the user’s question, the model is instructed to acknowledge the absence of supporting evidence rather than generating speculative or hallucinated information.

The retrieved document chunks preserve their original metadata throughout the prompt construction process. Consequently, the generated response can include references to the corresponding Technical Specification number, release version, section identifier, and page number.

This prompt engineering strategy enables the Telecom RAG Assistant to produce responses that are both conversational and technically reliable while maintaining complete traceability to the original engineering documentation.

7.16 LLM Response Generation

After the prompt has been constructed, it is transmitted to the selected Large Language Model (LLM) through the OpenRouter API. Unlike conventional conversational AI systems that rely solely on the model’s internal knowledge, the Telecom RAG Assistant generates responses exclusively from the contextual information retrieved from the indexed 3GPP Technical Specifications.

The prompt supplied to the LLM consists of three main components:

1. System instructions that define the assistant’s role and constrain its behaviour.
2. The retrieved specification chunks selected after semantic retrieval and Cross Encoder re-ranking.
3. The user’s original engineering question.

The system prompt explicitly instructs the model to answer only using the supplied engineering context. Whenever sufficient information cannot be found within the retrieved specification excerpts, the assistant is instructed to acknowledge the absence of evidence rather than generating speculative information. This prompt engineering strategy significantly reduces hallucination and improves the reliability of generated responses.

After inference, the generated answer is returned together with the supporting references corresponding to the retrieved document chunks. Each response therefore remains directly traceable to the original 3GPP specification, allowing engineers to verify every statement against the official standard.

Figure 7.6 illustrates a generated response describing the LTE Event A3 measurement event.

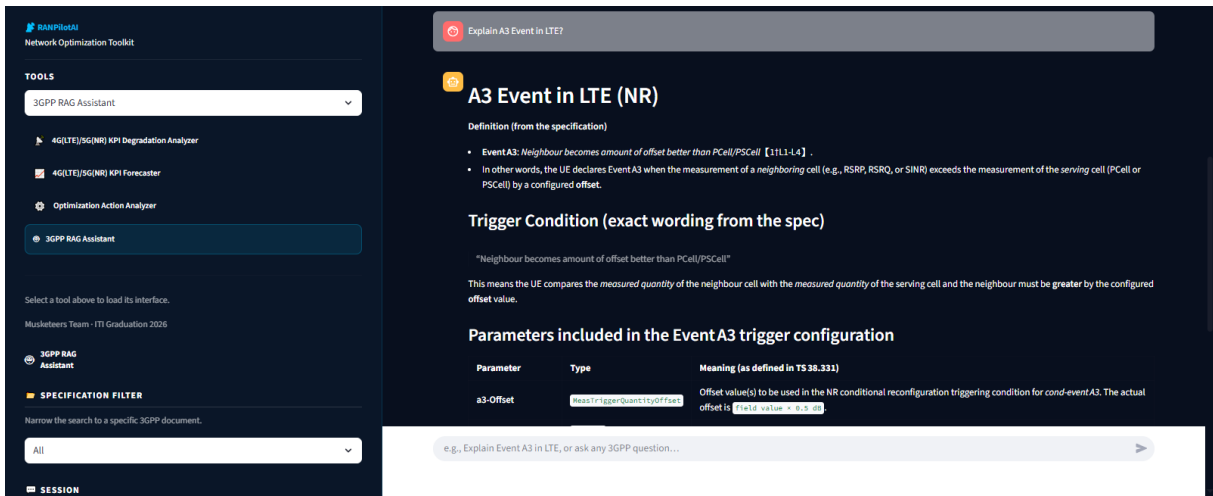


Figure 7.6: Generated engineering response for the LTE Event A3 measurement event.

The generated response contains both explanatory text and structured engineering information, allowing engineers to understand the underlying protocol behavior without manually navigating multiple specification sections.

7.17 User Interface

The Telecom RAG Assistant provides an interactive graphical user interface developed using Streamlit. The interface was designed to simplify interaction with complex engineering documentation while exposing the internal stages of the Retrieval-Augmented Generation pipeline.

The main interface consists of four principal components:

- Module navigation panel.
- Specification filtering controls.
- Conversational chat interface.
- Supporting citation viewer.

The conversational interface enables engineers to formulate technical questions using natural language instead of manually searching specification documents.

Figure 7.7 presents the main interface of the Telecom RAG Assistant.

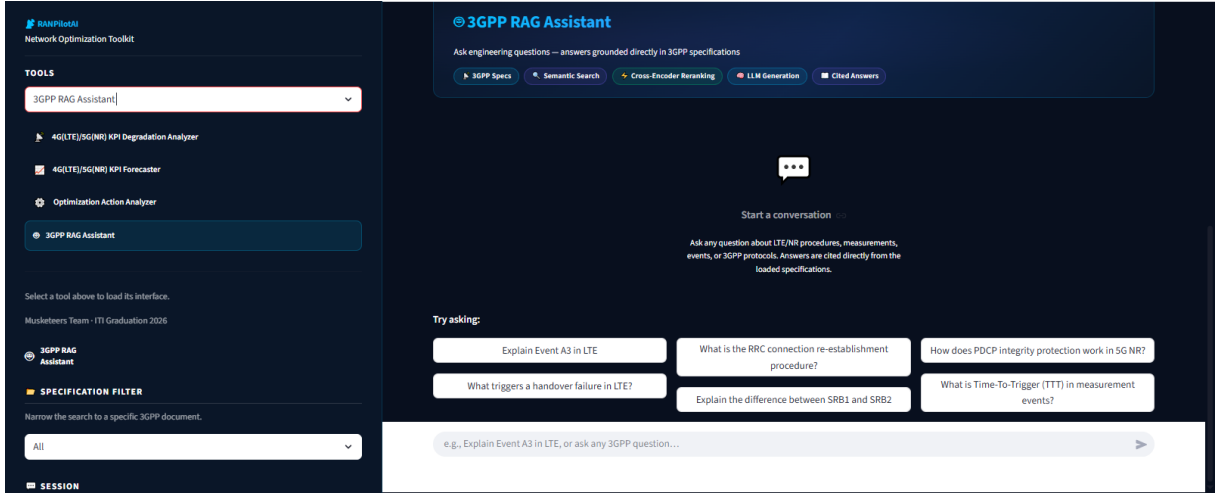


Figure 7.7: Main interface of the Telecom RAG Assistant.

One of the key capabilities of the developed Telecom RAG Assistant is its ability to maintain conversational context across successive user interactions. Rather than treating each query as an isolated request, the assistant leverages the previous questions and responses within the ongoing conversation to better interpret follow-up queries. This contextual awareness enables engineers to ask natural continuation questions without repeatedly providing the complete technical context, resulting in a more intuitive and efficient interaction. For example, after requesting an explanation of a specific LTE or NR procedure, the user may ask a follow-up question using pronouns or abbreviated terminology, and the assistant is able to infer the intended context from the conversation history before retrieving the most relevant specification excerpts. This conversational capability closely resembles the workflow of human technical discussions and significantly improves the usability of the system during troubleshooting and standards exploration. Figure 7.8

illustrates an example in which the assistant correctly interprets a follow-up question regarding the `H0-Conn` field by utilizing the context established in the previous interaction. The generated response remains grounded in the retrieved 3GPP specifications while preserving conversational continuity, allowing engineers to explore related concepts naturally without restarting the search process.

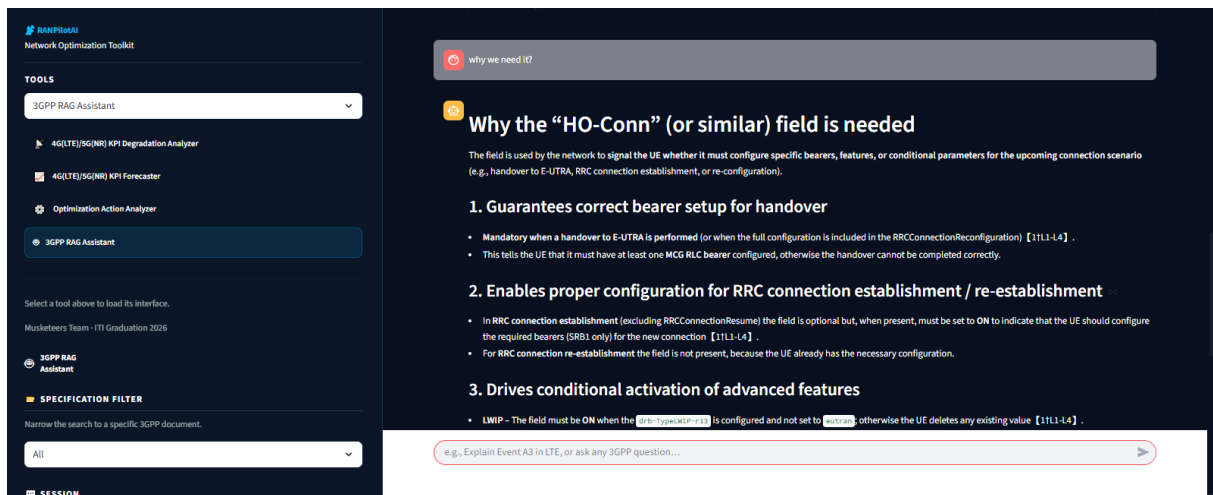


Figure 7.8: Context-aware response generated by the Telecom RAG Assistant using information from previous user interactions within the same conversation.

The assistant also produces concise engineering summaries highlighting the most important technical points extracted from the specifications.

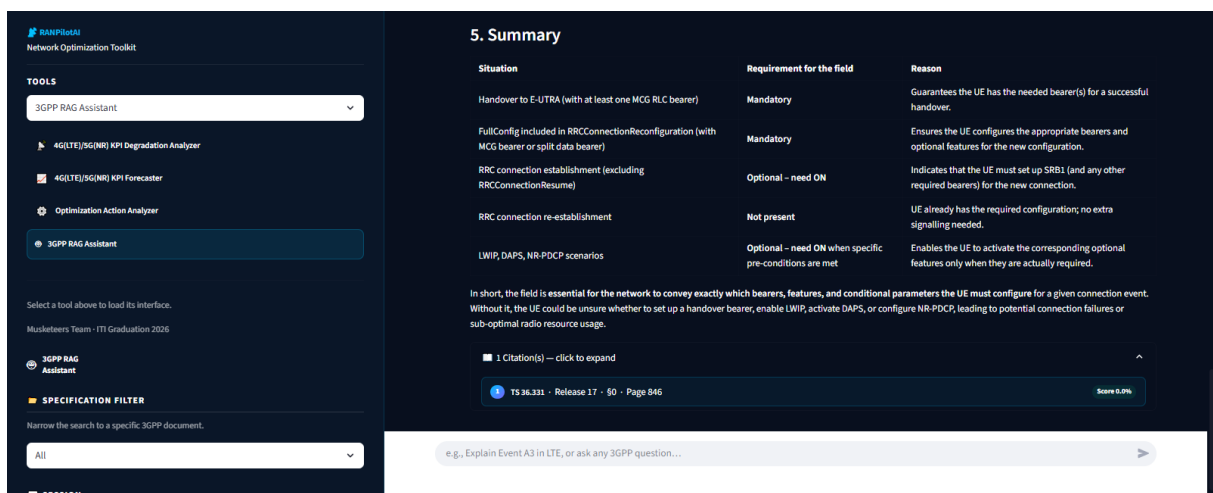


Figure 7.9: Structured engineering summary generated by the Telecom RAG Assistant.

The developed assistant is capable of interpreting engineering tables directly from the indexed specifications.

Figure 7.10 illustrates an example response describing the Packet Data Convergence Protocol (PDCP), while Figure 7.11 presents the corresponding engineering summary generated from the retrieved specification sections.

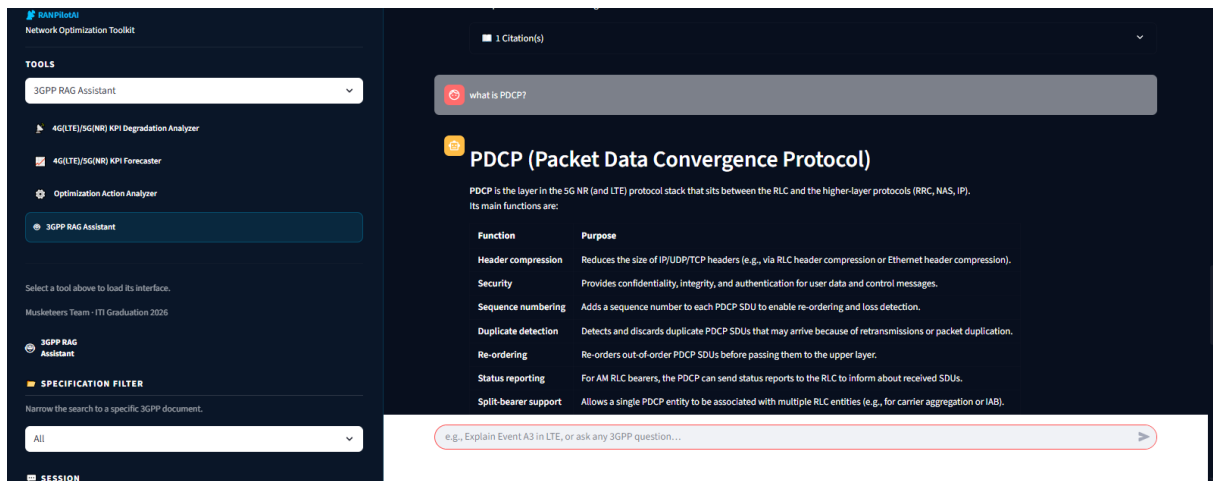


Figure 7.10: Generated response explaining the Packet Data Convergence Protocol (PDCP).

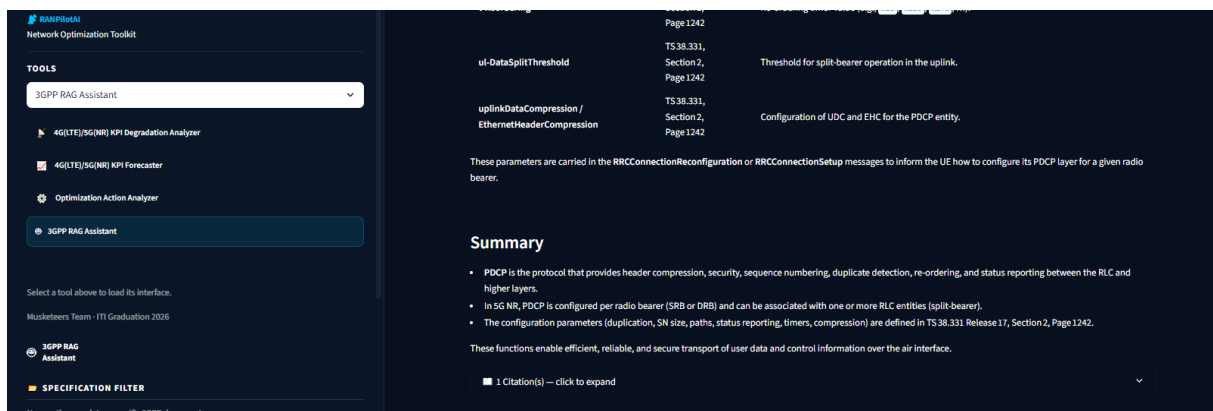


Figure 7.11: Structured engineering summary for the PDCP protocol.

The interface automatically attaches supporting citations to every generated response, allowing engineers to verify the retrieved information directly from the corresponding Technical Specification.

7.18 Performance Evaluation

The developed Telecom RAG Assistant incorporates an integrated profiling mechanism that records the execution time of every stage within the Retrieval-Augmented Generation pipeline. This profiling framework assists developers in identifying computational bottlenecks while providing transparency regarding the internal processing workflow.

For every submitted query, the profiler measures:

- Query embedding time.
- Vector database retrieval time.
- Cross Encoder re-ranking time.
- Prompt construction time.

- Large Language Model inference time.
- Total response latency.

Figure 7.12 illustrates the execution statistics generated during query processing.

```
mahmo@DESKTOP-RJT2JTQ MINGW64 /d/ITI_Graduation_project (main)
Network URL: http://192.168.1.8:8501

[14:59:27] Load Embedding Model | 3915.5 ms
[14:59:27] Init Embedder | 3918.9 ms
[14:59:27] Init Qdrant | 781.5 ms | path=D:\ITI_Graduation_project\RAG_tool\vector_db\qdrant_local
Qdrant collections: ['telecom_3gpp']
Points count: 2246
[14:59:27] Init OpenRouter LLM | 0.0 ms
[14:59:30] Load Reranker Model | 2941.0 ms | device=CPU
[14:59:30] Init Reranker | 2941.3 ms
[15:00:35] Query Embedding | 2149.2 ms
[15:00:35] Embedding Query | 2149.5 ms
[15:00:36] Qdrant Search | 28.7 ms | fetched=12, top_k=6
[15:00:36] Qdrant Retrieval | 29.0 ms | fetched=12
[15:00:36] Metadata Boost | 0.1 ms | query_terms_matched=1
[15:00:36] Remove Duplicates | 0.0 ms | input=12, output=3, removed=9
[15:00:36] Deduplicate | 0.3 ms | input=12, output=3
[15:00:36] Filter Chunks | 0.6 ms | original=3, filtered=2, removed=1
[15:00:36] Filter Context | 0.8 ms
[15:00:38] Reranking | 2720.7 ms | input=2, returned=2, predict_ms=2720.7, sort_ms=0.0, device=CPU
[15:00:38] Context Budget | 0.0 ms | selected=1, total_tokens=2516
[15:00:47] LLM Generate | 8343.6 ms
```

Figure 7.12: Pipeline profiling and execution statistics generated during query processing.

The collected execution statistics provide valuable insight into the computational cost of each pipeline stage and facilitate future optimization of the retrieval and inference process.

7.19 Discussion

The developed Telecom RAG Assistant successfully demonstrates how Retrieval-Augmented Generation can be adapted to the telecommunications domain through the integration of semantic retrieval, metadata-aware filtering, neural re-ranking, and conversational language models. By grounding every generated response in official 3GPP Technical Specifications, the system significantly reduces hallucination while maintaining complete traceability to the original engineering documentation.

Compared with traditional PDF searching, the proposed assistant provides a more intuitive interaction model that enables engineers to obtain accurate technical explanations using natural language. The modular architecture further simplifies future extensions, allowing additional specifications, embedding models, or language models to be incorporated with minimal modifications to the overall system.

7.20 Limitations

Although the developed Telecom RAG Assistant provides accurate and traceable engineering responses, several limitations remain.

The quality of generated responses depends directly on the availability and completeness of the indexed Technical Specifications. Information that has not been indexed cannot be retrieved by the system.

Additionally, the overall response latency is influenced by the selected embedding model, vector database search, Cross Encoder inference, and external Large Language Model response time. Processing very large document collections may therefore require additional optimization or distributed deployment strategies.

Finally, although Retrieval-Augmented Generation substantially reduces hallucination, complete elimination of incorrect responses cannot be guaranteed under all operating conditions.

7.21 Future Improvements

Several enhancements can further improve the proposed Telecom RAG Assistant.

Future work includes supporting additional telecommunication standards beyond LTE and 5G NR, integrating multimodal retrieval capable of interpreting engineering figures and diagrams, incorporating continuous indexing of newly released 3GPP specifications, and evaluating larger domain-specific embedding models.

Future versions may also integrate local Large Language Models to reduce dependency on external inference services while improving privacy and deployment flexibility.

7.22 Chapter Summary

This chapter presented the design and implementation of the Telecom RAG Assistant developed as part of the RANPilotAI platform. Beginning with the motivation and theoretical foundations of Retrieval-Augmented Generation, the chapter described the complete processing pipeline including PDF processing, section-aware chunking, embedding generation, vector storage, metadata filtering, hybrid semantic retrieval, Cross Encoder re-ranking, prompt construction, and language model inference. The implementation demonstrates how modern Artificial Intelligence techniques can be combined with official 3GPP Technical Specifications to provide an intelligent engineering assistant capable of answering complex telecommunication questions with high accuracy and complete source traceability. The presented architecture establishes a practical foundation for AI-assisted standards consultation and represents an important step toward intelligent engineering support systems for future mobile communication networks.

Chapter 8

System Integration

8.1 Introduction

The RANPilotAI platform represents the integration of the intelligent software modules developed throughout this project into a unified engineering environment for Radio Access Network (RAN) optimization. Instead of operating as independent applications, the developed modules are deployed within a single web-based platform that provides centralized navigation, shared software services, and a consistent graphical user interface.

The integrated platform enables RF engineers to perform multiple optimization tasks through a common environment. Network performance can be analyzed, future KPI behaviour can be predicted, optimization recommendations can be obtained, and official 3GPP Technical Specifications can be consulted without leaving the platform. This unified workflow improves engineering productivity while reducing the complexity associated with switching between multiple software tools.

From a software engineering perspective, the platform follows a modular architecture in which each analytical module encapsulates its own processing logic while sharing common infrastructure services. This design simplifies maintenance, improves scalability, and allows future analytical modules to be integrated with minimal architectural modifications.

This chapter describes the integration architecture adopted in the RANPilotAI platform, presents the overall organization of the developed modules, and explains the software engineering principles that guided the implementation of the integrated system.

8.2 Overall Platform Architecture

The RANPilotAI platform adopts a modular architecture in which the Telecom Hub functions as the central access point to all developed analytical modules. Rather than interacting directly with the individual processing engines, users communicate exclusively with the Telecom Hub, which manages application routing, session handling, and interaction with the underlying analytical components.

Figure [8.1](#) illustrates the overall architecture of the integrated platform.

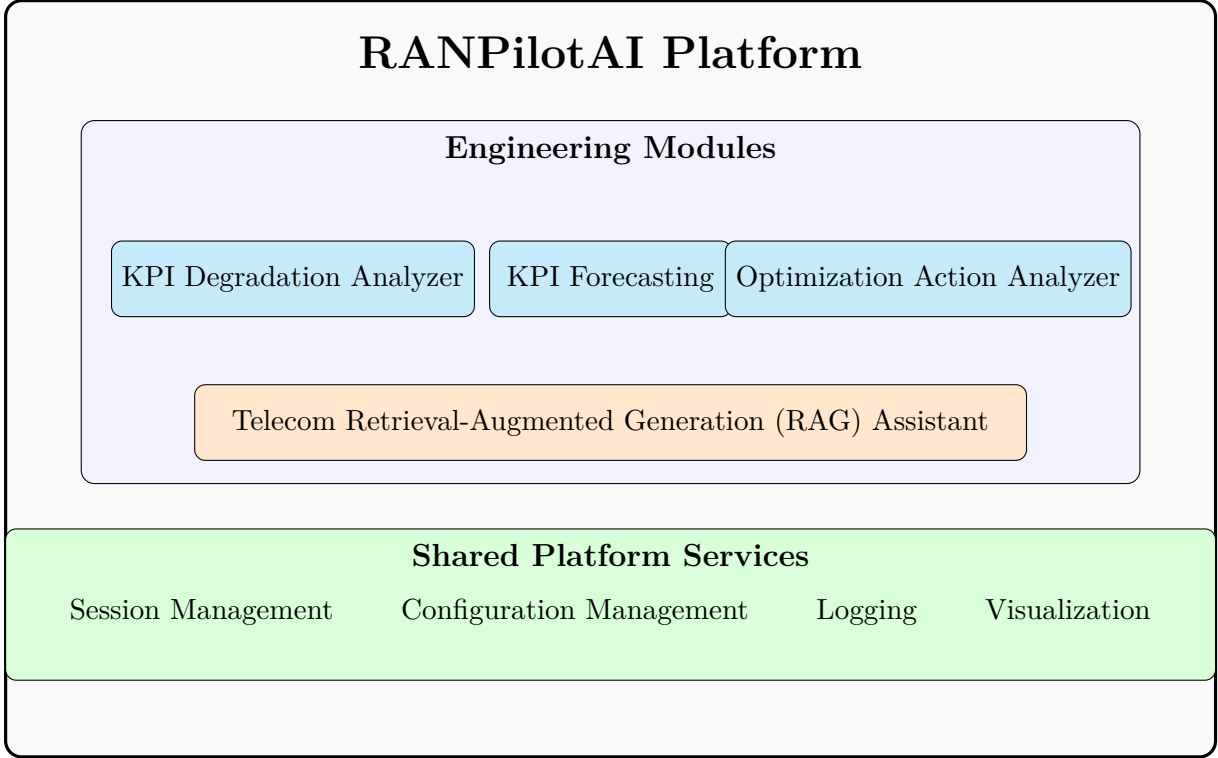


Figure 8.1: Overall architecture of the integrated RANPilotAI platform. The platform groups all analytical modules within a unified engineering environment while providing common platform services shared across the developed applications.

As illustrated in Figure 8.1, the RANPilotAI platform is organized around two principal components. The first component contains the engineering modules responsible for implementing the intelligent analytical capabilities of the platform, whereas the second component provides the shared platform services required by all modules. This organization separates domain-specific processing from common software infrastructure, thereby improving maintainability and facilitating future extensions.

Each analytical module executes independently while sharing common services such as session management, configuration handling, logging, and visualization. Consequently, software duplication is minimized and a consistent user experience is maintained across the entire platform.

8.3 Integration Objectives

The integration strategy adopted in the development of the RANPilotAI platform was guided by several software engineering objectives intended to improve the usability, maintainability, scalability, and extensibility of the system. Rather than focusing solely on integrating multiple analytical tools into a single application, the objective was to establish a modular software architecture capable of supporting future enhancements while maintaining a consistent engineering workflow [4].

The first objective was to provide a unified engineering environment that consolidates the complete RAN optimization workflow into a single platform. Instead of requiring engineers to execute separate applications for KPI analysis, forecasting, optimization planning, and standards consultation, all functionalities are accessible through the Telecom

Hub.

The second objective was to maximize software modularity. Every analytical component was developed as an independent subsystem responsible for a specific engineering task. This separation of responsibilities minimizes software dependencies, simplifies debugging, and enables individual modules to evolve independently without affecting the remaining components.

Another important objective was the reuse of common software services. Instead of duplicating functionalities such as session management, configuration loading, logging, visualization, and navigation within every module, these services are centralized and shared across the entire platform. This approach reduces software complexity while improving maintainability and consistency.

Finally, the architecture was designed with extensibility as a primary consideration. Future analytical modules, artificial intelligence models, or optimization algorithms can be incorporated into the existing framework by integrating them into the Engineering Layer without requiring modifications to the overall platform architecture.

8.4 Layered Software Architecture

From a software engineering perspective, the RANPilotAI platform follows a layered architecture that separates user interaction, application logic, engineering modules, and data management into distinct software layers. Such separation improves maintainability by ensuring that each layer is responsible for a well-defined set of functionalities while communicating only with adjacent layers.

Figure 8.2 presents the layered software architecture adopted in the implementation of the platform.

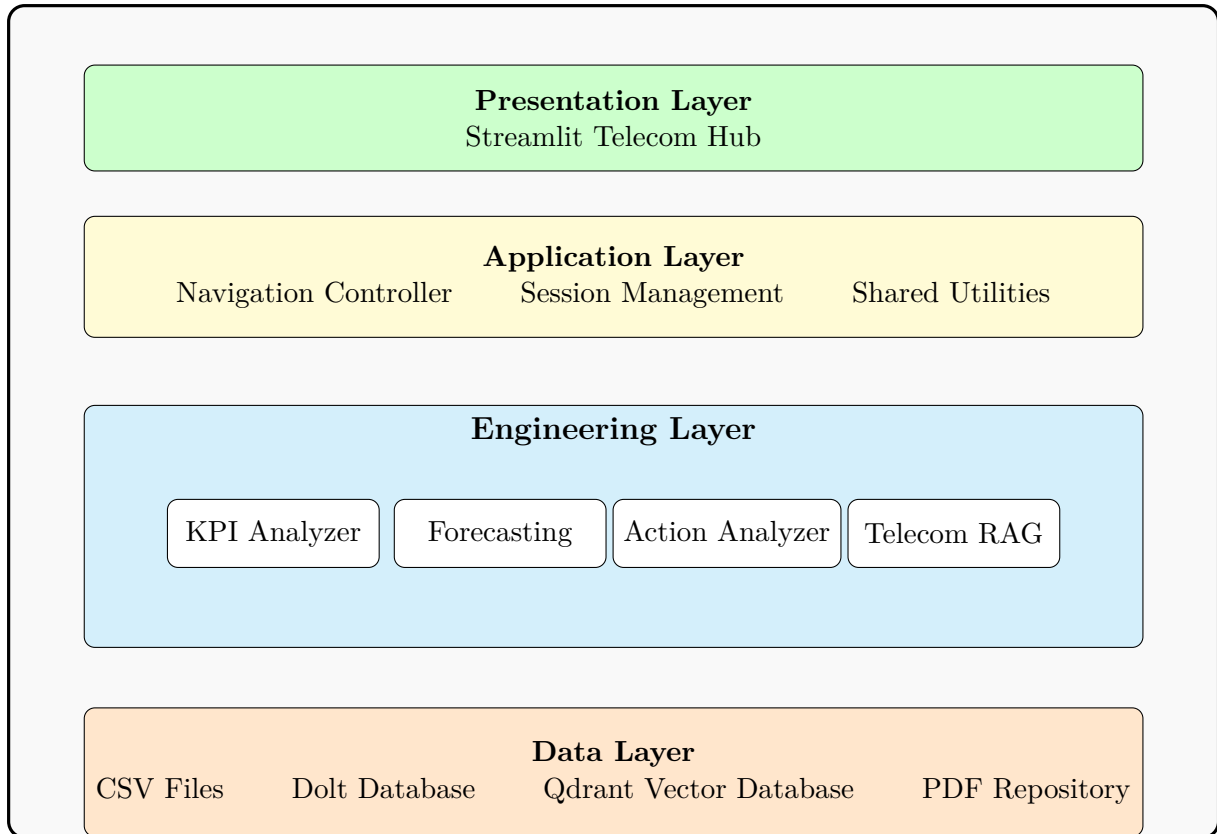


Figure 8.2: Layered software architecture of the RANPilotAI platform. The platform is organized into four logical layers separating presentation, application logic, engineering modules, and data management.

The layered architecture shown in Figure 8.2 separates the software into four logical layers, each responsible for a specific aspect of the platform. The Presentation Layer provides the graphical user interface through which engineers interact with the platform. The Application Layer manages navigation, user sessions, and shared software utilities that are reused across all analytical modules.

The Engineering Layer contains the four intelligent modules developed throughout this project. Each module executes independently while relying on the common services provided by the Application Layer. This separation ensures that modifications to one analytical component do not affect the remaining modules.

Finally, the Data Layer provides persistent storage and external resources required by the engineering modules. Structured datasets are maintained in CSV files, version-controlled engineering data are stored using Dolt, semantic document embeddings are managed by the Qdrant vector database, and the official 3GPP Technical Specifications are maintained within the PDF repository. This organization establishes a clear separation between data management and analytical processing while simplifying future maintenance and platform expansion [2].

8.5 Module Integration

The RANPilotAI platform integrates four independent analytical modules into a unified engineering environment while preserving the autonomy of each subsystem. Rather

than tightly coupling the modules, the integration strategy follows a modular design in which every component performs a dedicated engineering task and communicates with the Telecom Hub through standardized interfaces.

Each analytical module executes its own processing pipeline and maintains its own internal implementation. Consequently, modifications to one subsystem do not require changes to the remaining modules. This loose coupling improves software maintainability, facilitates debugging, and enables independent development of future platform extensions.

The KPI Degradation Analyzer is responsible for detecting abnormal network behaviour through KPI analysis. The KPI Forecasting module predicts future network performance using machine learning models. The Optimization Action Analyzer provides optimization recommendations based on historical engineering knowledge, while the Telecom Retrieval-Augmented Generation (RAG) Assistant enables engineers to interact with official 3GPP Technical Specifications using natural language.

Although these modules implement different analytical functionalities, they all follow the same execution model. Every request originates from the Telecom Hub, which identifies the requested service, forwards the request to the corresponding analytical module, receives the generated results, and presents them using a consistent graphical interface.

Figure 8.3 illustrates the interaction between the Telecom Hub and the developed analytical modules.

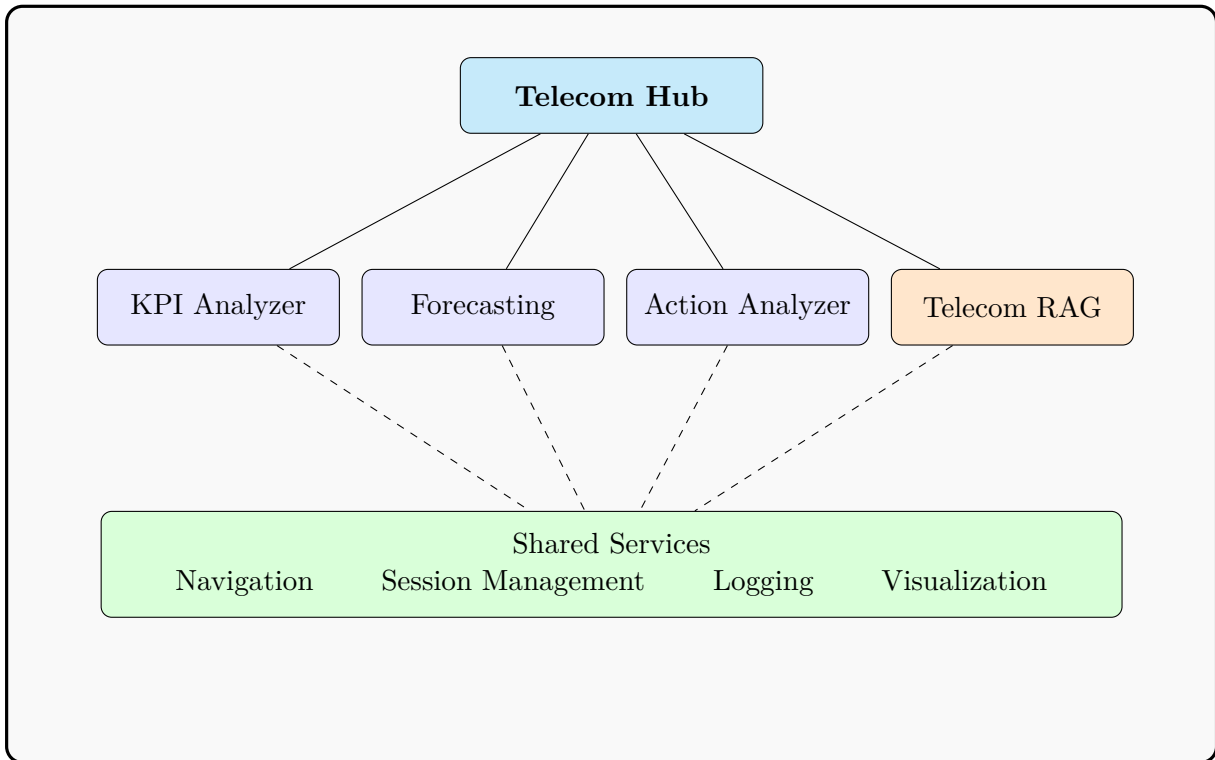


Figure 8.3: Module interaction within the RANPilotAI platform. The Telecom Hub coordinates user requests while common software services are shared among all analytical modules.

As illustrated in Figure 8.3, all user requests are processed through the Telecom Hub before reaching the corresponding analytical module. This centralized coordination simplifies application navigation while preserving complete independence between the engineering subsystems.

The shared service layer provides reusable software functionality required by every module, including session management, application navigation, logging, and visualization. Consequently, common software components are implemented only once, reducing redundancy and improving maintainability.

8.6 Shared Components

Although each analytical module implements a different engineering task, several software components are shared throughout the platform. Centralizing these functionalities simplifies software maintenance while ensuring a consistent user experience.

The principal shared components include:

- **Navigation Manager**

Responsible for routing users between the integrated analytical modules through the Telecom Hub.

- **Session Manager**

Maintains user selections, application state, and module-specific parameters throughout the execution session.

- **Configuration Manager**

Loads configuration files, model parameters, database connections, and application settings required by the analytical modules.

- **Logging Service**

Records application events and execution information to simplify debugging and future maintenance.

- **Visualization Engine**

Provides reusable plotting utilities and standardized graphical components that ensure consistent visualization across the platform.

Figure 8.4 presents the logical organization of the shared software components.

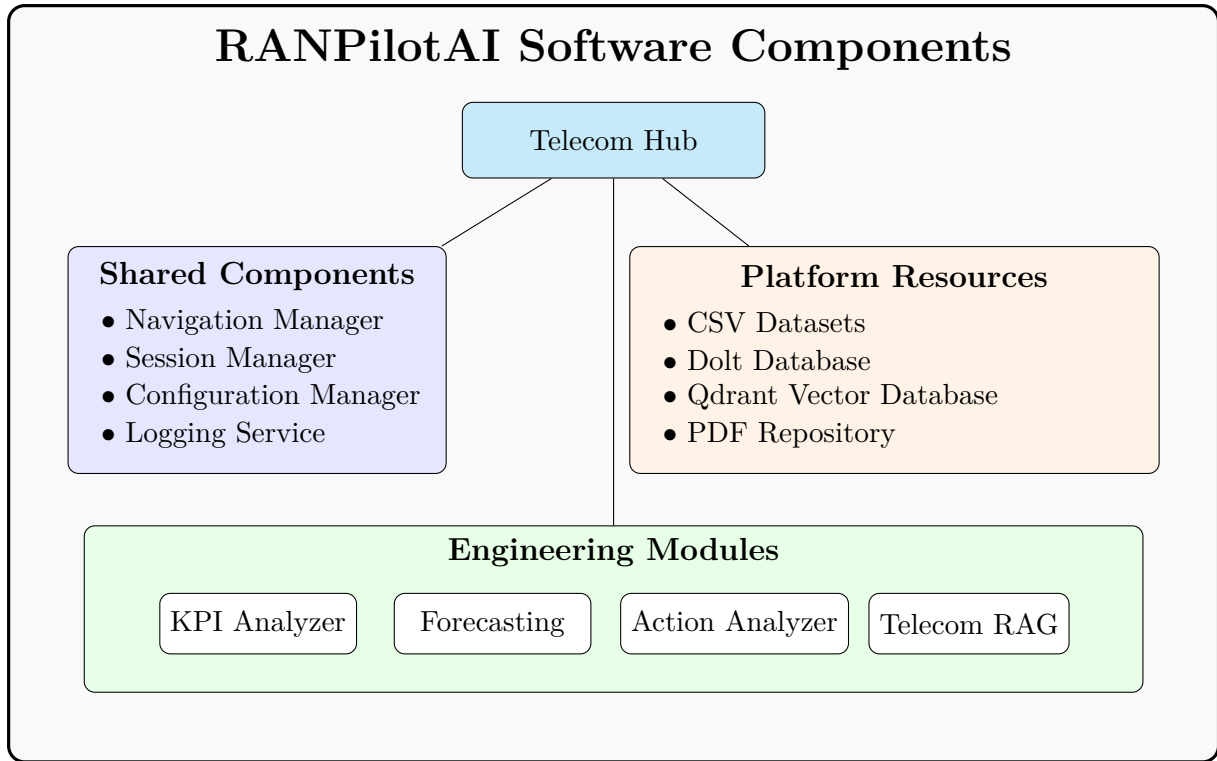


Figure 8.4: Component organization of the RANPilotAI platform showing the relationship between the Telecom Hub, shared software components, platform resources, and engineering modules.

Figure 8.4 illustrates the logical organization of the software components comprising the RANPilotAI platform. The Telecom Hub coordinates the interaction between users, shared software services, engineering modules, and platform resources. Shared components provide common functionality reused throughout the application, while the Engineering Modules implement the domain-specific analytical capabilities presented in the previous chapters.

Separating reusable software services from domain-specific modules significantly improves software maintainability by reducing duplication and establishing clear boundaries between infrastructure components and analytical functionality. Likewise, the explicit separation of platform resources enables individual modules to access common datasets, repositories, and vector databases without introducing unnecessary dependencies between the analytical subsystems.

8.7 Data Flow Between Modules

The analytical modules integrated within the RANPilotAI platform exchange information through a centralized execution model coordinated by the Telecom Hub. Instead of allowing direct communication between individual modules, every user request is first processed by the Telecom Hub, which determines the selected analytical service and forwards the request to the corresponding processing engine. This centralized data flow simplifies software maintenance, reduces coupling between modules, and establishes a consistent execution model throughout the platform.

Although each analytical module implements a different processing pipeline, all mod-

ules follow the same execution sequence. User inputs are collected through the graphical interface, processed by the selected analytical engine, and finally returned to the Telecom Hub for visualization. Depending on the selected module, additional resources such as CSV datasets, the Dolt database, the Qdrant vector database, or the PDF repository may be accessed during execution.

Figure 8.5 illustrates the overall data flow within the RANPilotAI platform.

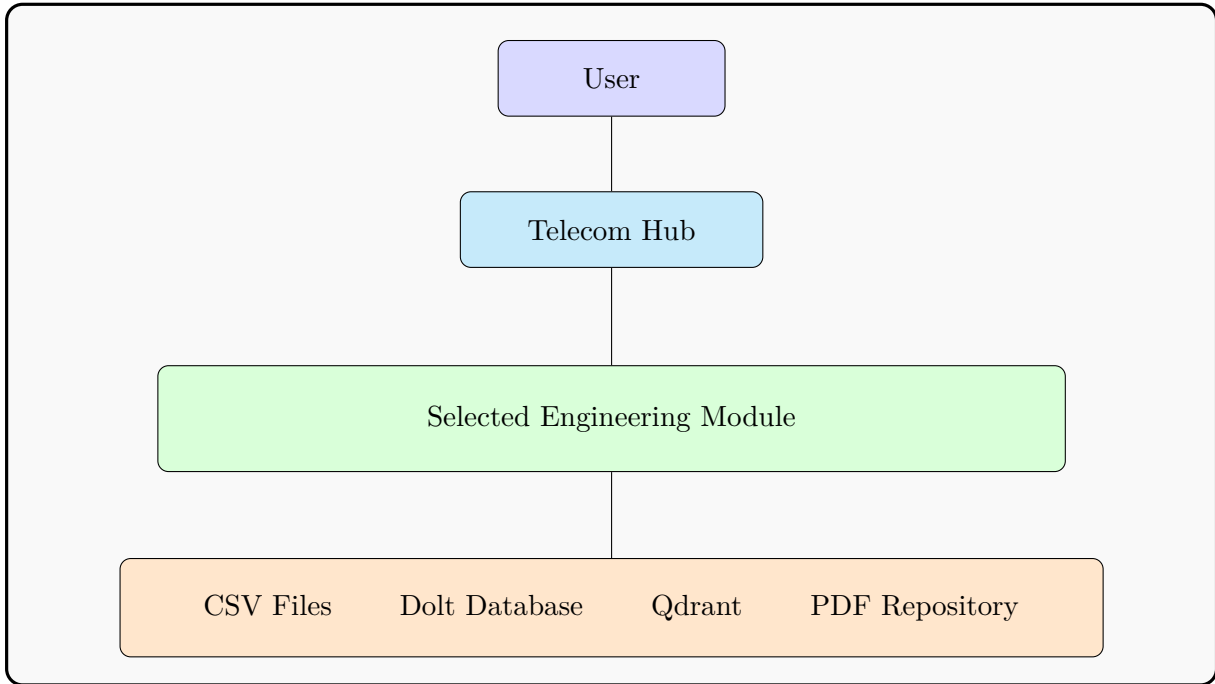


Figure 8.5: Overall data flow within the RANPilotAI platform. All requests are coordinated by the Telecom Hub before reaching the selected engineering module.

As shown in Figure 8.5, the Telecom Hub acts as the intermediary between the user and the analytical modules. Every engineering request follows the same execution model regardless of the selected module. This unified data flow simplifies the software architecture and enables the platform to incorporate additional analytical modules without modifying the existing execution framework.

8.8 User Navigation Workflow

The graphical user interface of RANPilotAI was designed to provide a simple and intuitive navigation experience. Rather than exposing the internal complexity of the analytical modules, users interact only with the Telecom Hub, which presents all available engineering tools through a unified interface.

Figure 8.6 illustrates the workflow followed by a typical user during interaction with the platform.

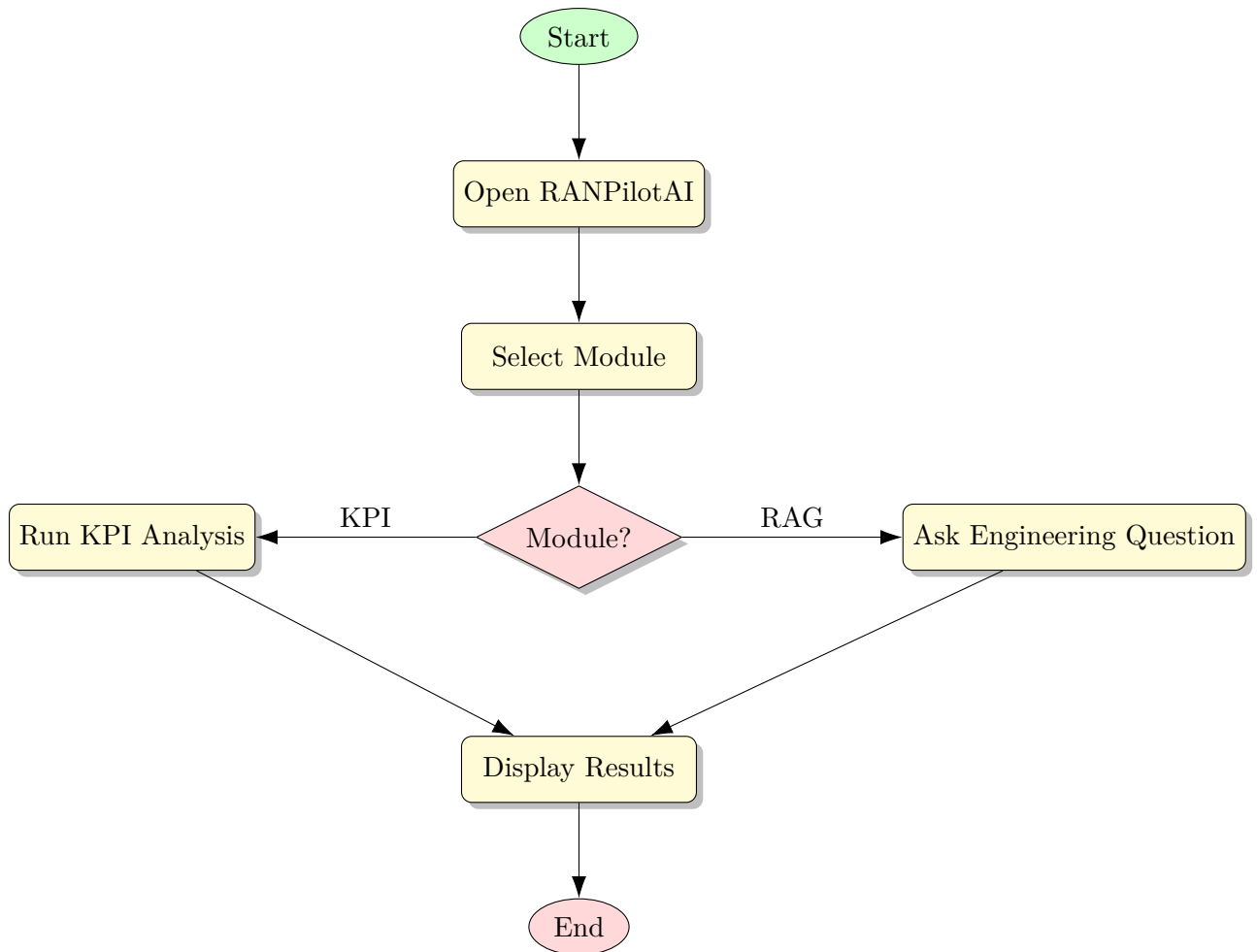


Figure 8.6: Typical user navigation workflow within the RANPilotAI platform.

The workflow begins when the user launches the platform and selects one of the available analytical modules. The Telecom Hub then forwards the request to the selected subsystem, waits for the analytical results, and presents them through the graphical interface. This interaction model remains identical regardless of the selected engineering module, thereby ensuring a consistent user experience throughout the platform.

8.9 Deployment Environment

The RANPilotAI platform was designed as a lightweight web-based application that can be deployed on a single workstation without requiring a distributed computing infrastructure. The deployment strategy prioritizes simplicity, portability, and ease of installation while providing sufficient computational resources for the analytical tasks implemented throughout the platform.

The Streamlit framework serves as the presentation layer of the application, providing an interactive graphical user interface through which users access the different engineering modules. Since Streamlit executes as a local web application, users can interact with the platform through a standard web browser without installing additional client software [5].

All engineering modules execute within the same Python runtime environment. This unified execution model simplifies dependency management, facilitates communication between modules, and enables the reuse of common software components. Furthermore,

executing the analytical modules locally reduces network latency and allows the platform to remain operational even in environments with limited Internet connectivity.

The platform relies on several local data resources. Structured engineering datasets are maintained as CSV files, historical engineering information is managed using the Dolt database, semantic document embeddings are stored within the Qdrant vector database, and official 3GPP Technical Specifications are maintained locally as PDF documents. These resources collectively provide the information required by the different analytical modules while remaining independent from external services.

External communication is required only during the execution of the Telecom Retrieval-Augmented Generation (RAG) Assistant. Once relevant engineering information has been retrieved from the local vector database, the generated prompt is forwarded to a Large Language Model (LLM) through the OpenRouter API to produce the final engineering response. Consequently, the majority of platform operations remain local while external communication is limited to natural language generation.

This deployment strategy provides an effective balance between computational efficiency and intelligent reasoning capabilities. Local execution ensures rapid analytical processing and secure management of engineering datasets, whereas cloud-based language models provide advanced natural language understanding without increasing the complexity of the local deployment.

8.10 Platform Scalability

Scalability was considered as one of the principal design objectives during the development of the RANPilotAI platform. Rather than implementing a monolithic software application, the platform adopts a modular architecture in which every analytical component operates as an independent subsystem connected through the Telecom Hub.

This modular organization enables future engineering modules to be incorporated without modifying the existing analytical components. New machine learning models, optimization algorithms, or AI assistants can be integrated by implementing the required processing logic and registering the module within the Telecom Hub navigation framework.

The shared software infrastructure further improves scalability by centralizing common functionalities such as session management, configuration loading, logging, visualization, and application routing. Since these services are reused by all analytical modules, the integration of additional functionalities does not introduce unnecessary software duplication.

The platform architecture also supports future migration toward distributed deployment environments. Although the current implementation executes on a single workstation, the modular software organization allows individual analytical modules to be deployed independently as web services or containerized applications. Such an architecture would enable computational workloads to be distributed across multiple processing nodes while preserving the unified user experience provided by the Telecom Hub.

Finally, the data management layer was designed to accommodate continuous expansion. Additional engineering datasets, telecommunications standards, optimization knowledge bases, and vector collections can be incorporated without requiring structural modifications to the software architecture. This capability ensures that the platform can evolve alongside future mobile communication technologies and emerging artificial intelligence techniques.

8.11 Integration Challenges

The integration of multiple analytical modules into a unified engineering platform introduced several software engineering challenges throughout the development process. Although each module was developed independently, integrating them into a single application required careful architectural decisions to ensure consistency, maintainability, and reliable execution.

One of the primary challenges involved coordinating heterogeneous analytical components. The developed modules employ different computational approaches, including traditional machine learning models, time-series forecasting algorithms, retrieval-based information systems, and Large Language Models. Establishing a unified execution model capable of supporting these diverse computational paradigms required the implementation of a common interaction framework through the Telecom Hub.

Another challenge concerned software dependency management. Since each analytical module relies on specialized Python libraries and external packages, maintaining compatibility between software components required careful version management and standardized project organization. Consolidating all dependencies into a common execution environment significantly simplified deployment while reducing software conflicts.

Maintaining a consistent graphical user interface also represented an important engineering consideration. Despite the diversity of analytical tasks performed by the individual modules, users should experience a unified interaction model throughout the platform. Standardizing visualization components, navigation mechanisms, and application layouts therefore became an essential aspect of the integration strategy.

Managing local and external resources presented an additional challenge. While most analytical operations rely exclusively on locally available datasets and vector databases, the Telecom RAG Assistant communicates with external language models through the OpenRouter API. Coordinating local retrieval with cloud-based response generation required careful management of network communication while preserving application responsiveness.

Finally, ensuring extensibility without sacrificing maintainability influenced many architectural decisions throughout the project. The adoption of modular software components, reusable utilities, and standardized interfaces enables future platform expansion while minimizing the impact of modifications on the existing system.

8.12 Future Extensions

Although the current implementation of the RANPilotAI platform provides a comprehensive intelligent environment for Radio Access Network (RAN) optimization, its modular software architecture enables several opportunities for future enhancement. The separation between the analytical modules and the Telecom Hub allows new functionalities to be incorporated without significant architectural modifications, ensuring that the platform can evolve alongside future telecommunications technologies.

One potential extension involves the integration of real-time network monitoring capabilities. Instead of relying solely on offline engineering datasets, future versions of the platform could establish direct communication with Operations Support Systems (OSS) or Network Management Systems (NMS) to analyze live network measurements. Such integration would enable continuous KPI monitoring and provide engineers with immediate analytical feedback based on current network conditions.

Another promising enhancement concerns the deployment architecture. Although the current implementation executes on a single workstation, the platform can be migrated to a cloud-native environment using containerization technologies such as Docker and orchestration platforms such as Kubernetes. This deployment model would improve scalability, simplify software maintenance, and enable concurrent access by multiple engineers.

The analytical capabilities of the platform can also be expanded by incorporating additional machine learning and artificial intelligence models. Future versions may include anomaly detection algorithms, reinforcement learning techniques for automatic optimization, predictive maintenance models, and advanced traffic forecasting approaches. Since each analytical component operates independently, these capabilities can be integrated while preserving the existing software architecture.

The Telecom Retrieval-Augmented Generation (RAG) Assistant can also be enhanced by integrating domain-specific language models trained exclusively on telecommunications literature and vendor documentation. Such models would improve the accuracy of technical responses and provide more specialized engineering recommendations. Furthermore, expanding the document repository to include vendor-specific implementation guides, operational manuals, and optimization procedures would significantly increase the knowledge available to the assistant.

Finally, future versions of RANPilotAI may provide collaborative engineering capabilities through multi-user authentication, centralized project management, shared engineering workspaces, and automated report generation. These features would transform the platform from an individual engineering assistant into a collaborative decision-support system capable of supporting larger operational teams.

8.13 Chapter Summary

This chapter presented the system integration strategy adopted during the development of the RANPilotAI platform. Unlike the previous chapters, which focused on the implementation of individual analytical modules, this chapter described how these independent components were combined into a unified engineering environment capable of supporting multiple Radio Access Network optimization tasks.

The chapter introduced the overall platform architecture, the layered software organization, the interaction between analytical modules, and the shared software components that provide common functionality across the platform. Furthermore, the deployment environment, scalability considerations, and software engineering challenges encountered during system integration were discussed in detail.

The modular architecture adopted throughout the development process establishes a flexible and extensible software foundation that simplifies future platform evolution while maintaining a consistent user experience. By integrating machine learning models, Retrieval-Augmented Generation techniques, optimization algorithms, and engineering knowledge within a single software environment, RANPilotAI provides a comprehensive intelligent platform for supporting modern Radio Access Network optimization activities.

The following chapter presents the overall conclusions of this work, summarizes the principal contributions of the project, and outlines potential directions for future research and development.

Chapter 9

Experimental Results

Write the chapter here.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This graduation project presented **RANPilotAI**, an integrated, AI-powered platform designed to automate and augment the daily workflows of Radio Frequency optimization engineers working in 4G LTE and 5G New Radio mobile networks. The platform addresses a fundamental gap in existing tooling: the absence of a unified, intelligent environment capable of covering the complete RAN optimization cycle — from performance degradation detection and trend forecasting, through configuration change impact assessment, to authoritative standards consultation.

The work delivered four specialized analytical modules, all integrated into a single web-based engineering environment called the **Telecom Hub**.

KPI Degradation Analyzer

The KPI Degradation Analyzer demonstrated that statistically rigorous automated detection of network performance degradation is achievable across a broad set of KPI categories without requiring manual threshold tuning or domain-specific scripting for each indicator. The three-layer pipeline — combining Welch’s t-test for statistical confidence scoring, RF-aware root cause inference with configurable severity weighting, and a data quality framework supporting sentinel-value filtering, unit-aware validation, and same-weekday baseline imputation — produced consistent and interpretable degradation reports across 13 KPI families including throughput, accessibility, retainability, mobility, availability, VoLTE, CSFB, and RRC re-establishment. Engineers can upload operator performance data and receive a structured root-cause summary within seconds, replacing a workflow that previously required significant manual effort.

KPI Forecasting Engine

The KPI Forecasting Engine demonstrated that combining multiple complementary forecasting models within a single interactive interface provides engineers with both predictive power and interpretability. The XGBoost model, built with leakage-safe cross-KPI feature engineering, captures non-linear relationships and inter-KPI dependencies that classical time-series methods cannot exploit. The Holt-Winters exponential smoothing model provides a transparent seasonal decomposition baseline whose assumptions are easier to audit in operational settings. Together with STL-based seasonality analysis, backtesting

metrics (MAE, RMSE, MAPE), recursive 7-day forward forecasts, and residual diagnostics, the engine enables engineers to anticipate network degradation before it manifests operationally, shifting the optimization workflow from reactive troubleshooting toward proactive planning.

Optimization Action Analyzer

The Optimization Action Analyzer addressed a long-standing challenge in RF optimization practice: the difficulty of establishing a causal connection between specific configuration changes and observed KPI impacts. By integrating Dolt, a version-controlled SQL database, the module provides a reproducible and auditable record of RF configuration history. Grouping same-day commits into unified action hunks, aligning configuration events with KPI time windows using weekday-matched baselines, and presenting results through interactive before/after overlay charts, delta bar plots, timeline views, and summary impact tables gave engineers a reliable, visual mechanism for attributing performance changes to individual optimization actions. This traceability significantly improves the quality of engineering decisions and supports knowledge transfer across operations teams.

Telecom RAG Assistant

The Telecom RAG Assistant demonstrated that Retrieval-Augmented Generation can be applied effectively to highly technical, standards-based engineering domains. The pipeline — comprising section-aware PDF chunking, BGE-large dense embeddings, local Qdrant vector search, cross-encoder re-ranking, and a strict anti-hallucination prompt — consistently produced traceable, citation-backed answers grounded directly in official 3GPP Technical Specifications. By returning the Technical Specification identifier, release version, section number, and page reference alongside every generated response, the assistant preserved the traceability that is essential in professional telecommunications engineering. Engineers were able to query hundreds of pages of protocol specifications in natural language and receive structured, verifiable answers within a fraction of the time required by traditional manual search.

System Integration

The integration of the four modules within the Telecom Hub validated the modular architecture adopted throughout the project. By separating domain-specific analytical logic from shared platform services — including session management, configuration handling, logging, and visualization — the platform achieved a consistent user experience across all modules while minimizing code duplication and simplifying maintenance. The Streamlit framework proved well-suited to rapid development of data-intensive engineering dashboards, providing a consistent layout model and interactive widget library that could be applied uniformly across all four analytical modules. The complete platform is implemented in Python 3.11 and is accessible through a standard web browser without client-side installation.

Overall Assessment

RANPilotAI successfully demonstrated that a small, cohesive software engineering team can design, implement, and integrate a production-quality AI platform for a specialized engineering domain within a single academic project cycle. The platform reduced the time required for the core activities of KPI performance analysis, trend forecasting, optimization impact assessment, and standards consultation, while maintaining engineering accuracy and full result traceability. The combination of statistical rigor, machine learning, retrieval-augmented generation, and version-controlled data management within a unified interface represents a meaningful step toward fully data-driven RAN optimization practice.

10.2 Limitations

Despite the results achieved, several limitations of the current implementation should be acknowledged.

- **Offline-only operation.** The platform currently operates on data uploaded manually by the engineer. There is no live integration with network management systems, Operations Support System (OSS) interfaces, or streaming data pipelines. Consequently, the platform supplements rather than replaces the data export step in the existing engineering workflow.
- **KPI scope.** The KPI Degradation Analyzer and Forecasting Engine were designed and validated against LTE performance data. While the analytical pipeline is applicable to 5G NR KPIs in principle, extended validation against 5G-specific indicators — including gNB-level throughput, PDCP layer metrics, and beam management KPIs — was not completed within the project timeline.
- **RAG knowledge base.** The Telecom RAG Assistant operates on a pre-indexed collection of 3GPP specifications. The accuracy and completeness of answers depend on which specifications have been indexed. Specifications not included in the local index cannot be referenced, and the knowledge base does not update automatically when new 3GPP releases are published.
- **LLM dependency.** The RAG assistant relies on an external Large Language Model for answer generation. The quality, latency, and availability of responses are therefore subject to the limitations and service conditions of the chosen LLM provider.
- **Forecasting horizon.** The KPI Forecasting Engine generates recursive 7-day forward forecasts. Forecast accuracy degrades over longer horizons due to error accumulation in the recursive multi-step prediction strategy, limiting the module's utility for long-term capacity planning.

10.3 Future Work

The current platform establishes a strong foundation for several meaningful extensions that would increase its operational value.

- **Live OSS integration.** Connecting the platform to a vendor OSS API or a streaming KPI data pipeline would eliminate the manual data upload step and enable near-real-time degradation detection. This would allow the KPI Degradation Analyzer to function as a continuous monitoring service rather than a batch-analysis tool.
- **5G NR KPI extension.** Extending the KPI schema, root-cause inference rules, and severity weighting system to cover 5G-specific KPIs — including gNB, DU, and CU layer indicators, beam management metrics, and network slicing performance dimensions — would broaden the platform’s applicability to next-generation network operations.
- **Automated action recommendation.** The Optimization Action Analyzer currently reports the measured impact of configuration changes but does not generate recommendations. Integrating a reinforcement learning or optimization model trained on the accumulated action-impact history stored in Dolt could produce proactive, data-driven optimization suggestions tailored to each cell’s historical behavior.
- **RAG knowledge base automation.** Building an automated pipeline that monitors 3GPP publication channels, downloads newly released or updated Technical Specifications, processes them through the chunking and embedding pipeline, and updates the Qdrant vector index would keep the RAG assistant current without manual intervention.
- **Multi-modal forecasting.** Incorporating external contextual features — such as public holiday calendars, major events, and weather data — as exogenous inputs to the XGBoost forecasting model could improve forecast accuracy for KPIs that exhibit strong event-driven demand patterns.
- **Role-based access and multi-tenancy.** Adding authentication, role-based access control, and multi-tenant data isolation would allow the platform to be deployed in a shared engineering environment where multiple optimization teams work concurrently without interference.
- **Exportable reports.** Extending the platform to generate structured PDF or HTML reports summarizing the outputs of each analytical module would integrate it more naturally into existing documentation and change management workflows used by operations teams.

10.4 Closing Remarks

The RANPilotAI project demonstrated that the convergence of classical statistical methods, modern machine learning, retrieval-augmented generation, and version-controlled data management — within a carefully integrated engineering platform — can substantially reduce the analytical burden placed on RF optimization engineers. As mobile networks grow in scale and complexity with the continued rollout of 5G New Radio and the emergence of Open RAN architectures, the need for intelligent, automated tooling will only increase.

The platform developed in this project is a practical step in that direction: a working, browser-accessible system that an engineer can deploy today to improve the speed,

consistency, and traceability of network optimization decisions. The modular architecture ensures that as new AI techniques and data sources become available, they can be incorporated without redesigning the platform from the ground up.

It is the hope of the development team that RANPilotAI, in its current form or as the foundation for future extensions, contributes meaningfully to the ongoing effort to make RAN optimization faster, smarter, and more accessible to the engineers who keep mobile networks running.

Chapter 11

User Manual

This chapter provides a complete user manual for the RANPilotAI platform. It covers system requirements, installation, launching the Telecom Hub, and step-by-step operating instructions for each of the four analytical modules.

11.1 System Requirements

- **Operating System:** Windows 10/11, Linux, or macOS
- **Python:** Version 3.11 or higher
- **RAM:** 8 GB minimum (16 GB recommended for large KPI datasets)
- **Disk Space:** At least 5 GB free (additional space required for 3GPP PDF files and the local vector database)
- **Internet Connection:** Required for the RAG Assistant (LLM API calls via Open-Router); all other modules operate fully offline
- **Web Browser:** Any modern browser (Chrome, Firefox, Edge) for the Streamlit interface

11.2 Installation

11.2.1 Step 1 — Install Python

Ensure Python 3.11 or higher is installed. Verify by running:

```
python --version
```

11.2.2 Step 2 — Install Dependencies

From the project root directory, install all required packages with a single command:

```
pip install -r requirements.txt
```

This installs all dependencies for all four tools, including Streamlit, Pandas, NumPy, SciPy, XGBoost, Statsmodels, Sentence Transformers, Qdrant, PyMuPDF, and python-docx.

11.2.3 Step 3 — Configure the RAG Assistant (First Time Only)

If you intend to use the Telecom RAG Assistant, an OpenRouter API key is required:

1. Copy the environment template:

```
cp RAG_tool/.env.example RAG_tool/.env
```

2. Open `RAG_tool/.env` and enter your OpenRouter API key:

```
OPENROUTER_API_KEY=your_api_key_here
```

3. Place your 3GPP specification PDF files in `RAG_tool/data/` using the naming convention:

```
TS_<NUMBER>_Rel_<RELEASE>.pdf
```

For example: `TS_36.331_Rel_17.pdf`

4. Run the ingestion pipeline to index the PDFs into the local vector database:

```
python -m app.ingest --all
```

Note: The first run downloads the BGE-large embedding model (≈ 1.3 GB).

11.2.4 Step 4 — Configure the Optimization Action Analyzer (If Required)

The Optimization Action Analyzer requires a running Dolt SQL server with two tables: `network_kpis` and `network_configs`. Install Dolt from <https://docs.dolthub.com/introduction/installation> and start the server on the default port before launching the platform.

11.3 Launching the Platform

The entire platform is launched through a single entry point from the project root directory:

```
streamlit run telecom_hub.py
```

After a few seconds, Streamlit will display a local URL (typically `http://localhost:8501`). Open this URL in your web browser. The Telecom Hub landing page will appear, displaying cards for all four analytical tools.

11.4 Navigating the Telecom Hub

The Telecom Hub interface consists of two areas:

- **Left Sidebar:** A dropdown menu listing all four tools. Select any tool to load it in the main area.
- **Main Content Area:** Displays the landing page or the selected tool's full interface.

To switch between tools, use the sidebar dropdown at any time. No page reload or server restart is required.

11.5 Tool 1 — KPI Degradation Analyzer

11.5.1 Purpose

Automatically detects performance degradation in LTE and 5G NR cells by comparing a recent KPI window against a historical baseline, ranking root causes, and generating exportable reports.

11.5.2 Input Data Format

The tool accepts Excel files (.xlsx or .xls) containing daily KPI exports. The file must include:

- An **eNodeB Name** column and a **Cell Name** (or **LocalCell Id**) column for cell identification
- A **Date** column in a standard date format
- One or more KPI counter columns (column names are matched flexibly using fuzzy matching)

11.5.3 Step-by-Step Usage

1. Select **KPI Degradation Analyzer** from the sidebar dropdown.
2. Click **Upload File** and select your Excel KPI export.
3. Select the appropriate worksheet from the sheet dropdown.
4. Choose the **KPI** to analyze from the KPI selector.
5. Set the **Comparison Days** (the number of recent days to compare against the baseline; default is 4).
6. Set the **Degradation Threshold** (the minimum degradation percentage to flag a cell; default is 30%).
7. Select the **Baseline Mode**:

- *Last Week* — compares the recent window against the same days in the previous week. Best for detecting sudden incidents.
 - *4-Week Rolling Median* — uses the median of the same weekdays over the past four weeks. Best for smoothing recurring weekly patterns.
8. Click **Run Selected KPI** to analyze one KPI, or **Analyze All KPIs** to process all 13 categories at once.
 9. Review the results table displaying degraded cells, degradation percentages, root cause categories, and recommended actions.
 10. Use the **Charts** tab to view bar charts of degraded cells per KPI and root cause distribution.
 11. Use the **Trends** tab to view before/after line chart overlays.
 12. Export results using the **Save CSV** button or generate a formatted Word report with **Generate Report**.

11.5.4 Understanding the Output

- **rf_severity**: Operational RF impact label — Normal, Medium, High, or Critical.
- **rca_pattern**: Root cause pattern — Outage, Congestion, Radio Quality, Coverage, Interference, Mobility, Demand, or Unknown.
- **main_root_cause_category**: The highest-scored root cause category.
- **main_recommended_action**: The engineering action recommended for the top cause.
- **analysis_confidence**: Confidence label based on statistical significance, data completeness, and severity.

11.6 Tool 2 — KPI Forecasting Engine

11.6.1 Purpose

Forecasts future KPI trends using machine learning and statistical time-series models, enabling engineers to anticipate degradation before it occurs operationally.

11.6.2 Input Data Format

A CSV file with:

- A **Date** column in YYYY-MM-DD format
- A **Cell Name** column
- One column per KPI using the raw counter names defined in `config.py`

11.6.3 Step-by-Step Usage

1. Select **KPI Forecaster** from the sidebar dropdown.
2. Upload your cleaned KPI CSV file when prompted.
3. From the sidebar, choose a **Forecasting Method**: XGBoost, Holt-Winters, or Compare Both.
4. Select the **KPI to Forecast** from the dropdown (12 KPIs supported).
5. Select a **Cell** from the dropdown (sorted by weekly seasonality strength, strongest first).
6. Set the **Hold-out Test Days** (the number of most recent days used for backtesting; default is 4).
7. Optionally enable:
 - *Show Next-7-Day Future Forecast* — extends the prediction beyond the test window.
 - *Flag Seasonality Strength* — displays STL-based seasonality badges.
 - *Compare vs Naive Baseline* — benchmarks models against naive and weekly-mean baselines.
8. Adjust XGBoost or Holt-Winters hyperparameters from the sidebar if required.
9. Review the forecast chart, accuracy metrics (MAE, RMSE, MAPE), predictions table, and the 7-day forward forecast table.
10. Expand the **Feature Importance** panel to inspect the top-15 XGBoost predictors.
11. Expand the **Residual Analysis** panel for diagnostic statistics (Durbin-Watson, Ljung-Box).
12. Use the **LLM-Ready Report** panel to generate a structured summary suitable for pasting into an LLM for further optimization reasoning.

11.7 Tool 3 — Optimization Action Analyzer

11.7.1 Purpose

Correlates RF configuration changes (antenna tilt adjustments, power parameter updates) recorded in a Dolt version-controlled database with before/after KPI impact, enabling engineers to quantitatively evaluate the effect of optimization actions.

11.7.2 Prerequisites

- Dolt installed and a Dolt SQL server running on the default port (3306)
- A Dolt database containing `network_kpis` and `network_configs` tables
- Configuration changes committed to the Dolt database (each commit corresponds to one optimization action)

11.7.3 Step-by-Step Usage

1. Select **Optimization Action Analyzer** from the sidebar dropdown.
2. The tool connects to the Dolt server automatically and reads the commit log.
3. Select an **Action Hunk** (a date grouping of one or more Dolt commits) from the timeline to analyze.
4. For each KPI detected in the database, set the **polarity**:
 - *Higher is Better* (e.g., Throughput, Setup Success Rate)
 - *Lower is Better* (e.g., Drop Rate, Block Rate)
5. Review the results:
 - **Action Timeline** — a visual chart of all Dolt commits.
 - **Before/After Line Chart** — overlays the same weekday before and after the action.
 - **Delta Bar Chart** — shows the absolute KPI change per time interval, color-coded green (improved) or red (degraded).
 - **KPI Summary Table** — structured before/after comparison with absolute delta, percentage change, and status.

11.8 Tool 4 — Telecom RAG Assistant

11.8.1 Purpose

Answers engineering questions about LTE and 5G NR standards in natural language, with every response grounded in and cited from locally indexed 3GPP Technical Specification PDFs.

11.8.2 Prerequisites

Ensure that the RAG Assistant was configured as described in Step 3 of the Installation section: the `.env` file must contain a valid OpenRouter API key, 3GPP PDFs must be placed in `RAG_tool/data/`, and the ingestion pipeline must have been run at least once.

11.8.3 Step-by-Step Usage

1. Select **Telecom RAG Assistant** from the sidebar dropdown.
2. Optionally use the **Specification Filter** in the sidebar to restrict answers to a specific TS number (e.g., `36.331`) or 3GPP release (e.g., `17`).
3. Type your engineering question in natural language in the chat input box. Examples:
 - “What are the trigger conditions for Event A3 in LTE RRC?”
 - “What is the purpose of the RACH procedure in 5G NR?”

- *“Explain the difference between RLC AM and UM modes.”*
4. Press Enter or click Send.
 5. The assistant will display a structured technical response followed by expandable citation panels showing the exact TS number, release version, section number, and page number for each retrieved chunk.
 6. If the answer cannot be found in the indexed specifications, the assistant will explicitly state so rather than generating an unverified response.

11.8.4 Performance Tuning

The retrieval behaviour can be adjusted via the `RAG_tool/.env` file:

- `TOP_K` (default: 5) — number of specification chunks supplied to the language model.
- `FETCH_MULTIPLIER` (default: 4) — multiplier for chunks retrieved from Qdrant before re-ranking.
- `CHUNK_SIZE` (default: 800 tokens) — maximum size of each indexed chunk.
- `DEBUG_MODE` (default: false) — enables pipeline stage timing and prompt statistics in the sidebar.

11.9 Troubleshooting

The platform does not start. Verify that Python 3.11 or higher is installed and that all dependencies were installed with `pip install -r requirements.txt`. Check that `telecom_hub.py` is executed from the project root directory.

The KPI Degradation Analyzer cannot find KPI columns. Ensure your Excel file includes column headers that partially match the expected counter names. The tool uses fuzzy matching, but column names must be recognizable. Refer to the KPI configuration reference in `KPI_and_its_related_counters.md`.

The Forecaster produces a flat or unstable forecast. This may occur when the data window is shorter than 21 days, causing lag features to be sparsely populated. Reduce `max_depth` and `n_estimators` in the XGBoost sidebar, or switch to the Holt-Winters model.

The Optimization Action Analyzer cannot connect to Dolt. Verify that the Dolt SQL server is running (`dolt sql-server`) and that the `network_kpis` and `network_configs` tables exist in the active database.

The RAG Assistant returns “I cannot find this in the supplied specifications.” The queried concept may not be present in the indexed PDFs. Verify that the relevant 3GPP specification has been placed in `RAG_tool/data/` and re-run the ingestion pipeline (`python -m app.ingest -all`).

The RAG Assistant is slow to respond. Response latency depends on the OpenRouter API. Enable `DEBUG_MODE=true` in `.env` to display per-stage pipeline timing in the sidebar and identify bottlenecks.

Bibliography

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [2] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Prentice Hall, 2023.
- [3] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [4] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
- [5] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill, 2014.
- [6] E. Dahlman, S. Parkvall, and J. Sköld, *5G NR: The Next Generation Wireless Access Technology*, 2nd ed. Academic Press, 2020.
- [7] S. Sesia, I. Toufik, and M. Baker, *LTE — The UMTS Long Term Evolution: From Theory to Practice*, 2nd ed. Wiley, 2011.
- [8] E. Alpaydin, *Introduction to Machine Learning*, 4th ed. MIT Press, 2020.
- [9] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [10] D. C. Montgomery and G. C. Runger, *Applied Statistics and Probability for Engineers*, 7th ed. Wiley, 2018.
- [11] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. Springer, 2009.
- [12] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. OTexts, 2018.
- [13] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: With Applications in R*, 2nd ed. Springer, 2021.
- [14] H. Holma and A. Toskala, *LTE for UMTS: Evolution to LTE-Advanced*, 2nd ed. Wiley, 2011.
- [15] A. R. Mishra, *Advanced Cellular Network Planning and Optimisation: 2G/2.5G/3G Evolution to 4G*. Wiley, 2007.
- [16] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.

- [17] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed. Wiley, 2017.
- [18] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. SAGE Publications, 2017.
- [19] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [20] T. S. Rappaport, R. W. H. Jr., R. C. Daniels, and J. N. Murdock, *Millimeter Wave Wireless Communications*. Pearson, 2024.